

15. ACPI Source Language (ASL) Reference

This section formally defines the ACPI Control Method Source Language (ASL). ASL is a source language for writing ACPI control methods. OEMs and BIOS developers write control methods in ASL and then use a translator tool (compiler) to generate ACPI Machine Language (AML) versions of the control methods. For a formal definition of AML, see the ACPI Control Method Machine Language (AML) Specification, section 16.

AML and ASL are *different languages* though they are closely related.

Every ACPI-compatible OSes must support AML. A given user can define some arbitrary source language (to replace ASL) and write a tool to translate it to AML.

An OEM or BIOS vendor needs to write ASL and be able to single step AML for debugging. (Debuggers and similar tools are expected to be AML level tools, not source level tools.) An ASL translator implementer must understand how to read ASL and generate AML. An AML interpreter author must understand how to execute AML.

This section has two parts:

- The ASL grammar, which is the formal ASL specification and also serves as a quick reference.
- A full ASL reference, which repeats the ASL term syntax and adds information about the semantics of the language.

15.1 ASL Language Grammar

The purpose of this section is to state unambiguously the grammar rules used by the syntax checker of an ASL compiler.

ASL statements declare objects. Each object has three parts, two of which can be null.

```
Object := ObjectType FixedList VariableList
```

FixedList refers to a list, of known length, that supplies data that all instances of a given **ObjectType** must have. A fixed list is written as (*a* , *b* , *c* , ...) where the number of arguments depends on the specific **ObjectType**, and some elements can be nested objects, that is (*a* , *b* , (*q* , *r* , *s* , *t*) , *d*). Arguments to a **FixedList** can have default values, in which case they can be skipped. Thus, (*a* , *c*) will cause the default value for the second argument to be used. Some **ObjectTypes** can have a null **FixedList**, which is simply omitted. Trailing arguments of some object types can be left out of a fixed list, in which case the default value is used.

VariableList refers to a list, not of predetermined length, of child objects that help define the parent. It is written as { *x* , *y* , *z* , *aa* , *bb* , *cc* } where any argument can be a nested object. **ObjectType** determines what terms are legal elements of the **VariableList**. Some **ObjectTypes** may have a null variable list, which is simply omitted.

Other rules for writing ASL statements are the following:

- Multiple blanks are the same as one. Blank, (,) , ‘ , ’ and newline are all token separators.
- // marks the beginning of a comment, which continues from the // to the end of the line.
- /* marks the beginning of a comment, which continues from the /* to the next */.
- “ ” surround an ASCII string.
- Numeric constants can be written in two ways: ordinary decimal, or hexadecimal, using the notation 0xdd
- **nothing** indicates an empty item. For example { **nothing** } is equivalent to { }

15.1.1 ASL Grammar Notation

The notation used to express the ASL grammar is specified in the following table.

Table 15-1 ASL Grammar Notation

Notation Convention	Description	Example
Term := Term Term ...	The term to the left of := can be expanded into the sequence of terms on the right.	aterm := bterm cterm means that aterm can be expanded into the two-term sequence of bterm followed by cterm.
Angle brackets (< >)	Used to group items.	<a b> <c d> means either a b or c d.
Bar symbol ()	Separates alternatives.	aterm := bterm <cterm dterm> means the following constructs are possible: bterm cterm dterm aterm := <bterm cterm> dterm means the following constructs are possible: bterm dterm cterm dterm
Term Term Term	Terms separated from each other by spaces form an ordered list.	N/A.
Word in bold.	Denotes the name of a term in the ASL grammar, representing any instance of such a term.	In the following ASL term definition: ThermalZone (ZoneName) {NamedObjectList} the item in bold is the name of the term.
Word in italics	Names of arguments to objects that are replaced for a given instance.	In the following ASL term definition: ThermalZone (ZoneName) {NamedObjectList} the italicized item is an argument. The item that is not bolded or italicized is defined elsewhere in the ASL grammar.
Single quotes (‘ ’)	Indicate constant characters.	‘A’
0xdd	Refers to a byte value expressed as 2 hexadecimal digits.	0x21 means a value of hexadecimal 21, or decimal 37. Note that a value expressed in hexadecimal must start with a leading zero (0).
Dash character (-)	Indicates a range.	1-9 means a single digit in the range 1 to 9 inclusive.

15.1.2 ASL Names

LeadNameChar	:= 'A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I' 'J' 'K' 'L' 'M' 'N' 'O' 'P' 'Q' 'R' 'S' 'T' 'U' 'V' 'W' 'X' 'Y' 'Z' '\'
NameChar	:= '0' '1' '2' '3' '4' '5' '6' '7' '8' '9' LeadNameChar
RootChar	:= '\'
NameSeg	:= <LeadNameChar NameChar NameChar NameChar> <LeadNameChar NameChar NameChar> <LeadNameChar NameChar> <LeadNameChar>
NameString	:= <RootChar NamePath> <PrefixPath NamePath>
PrefixPath	:= Nothing <'^' PrefixPath>
NamePath	:= Nothing <NameSeg NamePathTail>
NamePathTail	:= Nothing <\'.' NameSeg NamePathTail>

15.1.3 ASL Language and Terms

ASLCode	:= DefinitionBlockTerm
DefinitionBlockTerm	:= DefinitionBlock (<i>AMLFileName</i> , //String <i>TableSignature</i> , //String <i>ComplianceRevision</i> , //ByteConst <i>OEMID</i> , //String <i>TableID</i> , //String <i>OEMRevision</i> , //DWordConst) {TermList}
TermList	:= Nothing <Term TermList>
Term	:= Object Type1Opcode Type2Opcode
CompilerDirective	:= IncludeTerm ExternalTerm
ObjectList	:= Nothing <Object ObjectList>
Object	:= CompilerDirective NamedObject NameSpaceModifier UserTerm
DataObject	:= BufferTerm PackageTerm LiteralData DataMacros
LiteralData	:= Integer String ConstTerm
ComputationalData	:= Integer String BufferTerm
DataMacros	:= EISAIDTerm ResourceTemplateTerm
NamedObject	:= BankFieldTerm CreateBitFieldTerm CreateByteFieldTerm CreateWordFieldTerm CreateFieldTerm CreateWordFieldTerm DeviceTerm EventTerm FieldTerm IndexFieldTerm MethodTerm MutexTerm OpRegionTerm PowerResTerm ProcessorTerm ThermalZoneTerm
NameSpaceModifier	:= AliasTerm NameTerm ScopeTerm
UserTerm	:= NameString(//NameString=>MethodTerm ArgList) => DataObject
ArgList	:= Nothing <TermArg ArgListTail>
ArgListTail	:= Nothing <',' TermArg ArgListTail>
TermArg	:= Type2Opcode DataObject UserTerm ArgTerm LocalTerm
Type1Opcode	:= BreakTerm BreakPointTerm FatalTerm IfElseTerm LoadTerm NoOpTerm NotifyTerm ReleaseTerm ResetTerm ReturnTerm SignalTerm SleepTerm StallTerm UnloadTerm WhileTerm // A Type1OpCode term can only be used standing alone on a // line of ASL code; because these types of terms do not // return a value so they cannot be used as a term in an // expression.

```

Type2Opcode      := AcquireTerm | AddTerm | AndTerm | ConcatTerm | CondRefOfTerm
                  | DecTerm | DerefOfTerm | DivideTerm | FindSetLeftBitTerm | |
                  | FindSetRightBitTerm | FromBCDTerm | IncTerm | IndexTerm |
                  | LAndTerm | LEqualTerm | LGreaterTerm | LGreaterEqualTerm |
                  | LLessTerm | LLessEqualTerm | LNotTerm | LNotEqualTerm |
                  | LOrTerm | MatchTerm | MultiplyTerm | NAndTerm | NORTerm |
                  | NotTerm | ObjectTypeTerm | OrTerm | RefOfTerm |
                  | ShiftLeftTerm | ShiftRightTerm | SizeOfTerm | StoreTerm |
                  | SubtractTerm | ToBCDTerm | WaitTerm | XorTerm
                  // A Type2Opcode term returns a value that can be used in
                  // an expression.

IncludeTerm      := Include(
                    IncFilePathName           //String
                  )

ExternalTerm     := External(
                    ObjName,                   //NameString
                    ObjType                    //Nothing | ObjectTypeKeyword
                  )

BankFieldTerm    := BankField(
                    RegionName,                //NameString
                    BankName,                  //NameString
                    BankValue,                 //TermArg=>DWordConst
                    AccessType,                //AccessTypeKeyword
                    LockRule,                  //LockRuleKeyword
                    UpdateRule                 //UpdateRuleKeyword
                  ) {FieldUnitList}

FieldUnitList    := Nothing | <FieldUnit FieldUnitListTail>
FieldUnitListTail := Nothing | <',' FieldUnit FieldUnitListTail>

FieldUnit        := FieldUnitEntry | OffsetTerm | AccessAsTerm
FieldUnitEntry   := <Nothing | NameSeg> ',' Integer

OffsetTerm       := Offset(
                    ByteOffset                 //Integer
                  )

AccessAsTerm     := AccessAs(
                    AccessType,                //AccessTypeKeyword
                    AccessAttribute            //Nothing | ByteConst
                  )

CreateBitFieldTerm := CreateBitField(
                    SourceBuffer,              //TermArg=>BufferTerm
                    BitIndex,                  //TermArg=>Integer
                    BitFieldName               //NameString
                  )

CreateByteFieldTerm := CreateByteField(
                    SourceBuffer,              //TermArg=>BufferTerm
                    ByteIndex,                 //TermArg=>Integer
                    ByteFieldName              //NameString
                  )

CreatedWordFieldTerm := CreatedWordField(
                    SourceBuffer,              //TermArg=>BufferTerm
                    ByteIndex,                 //TermArg=>Integer
                    DWordFieldName             //NameString
                  )

CreateFieldTerm  := CreateField(
                    SourceBuffer,              //TermArg=>BufferTerm
                    BitIndex,                  //TermArg=>Integer
                    NumBits,                   //TermArg=>Integer
                    FieldName                  //NameString
                  )

```

```

CreateWordFieldTerm      := CreateWordField(
                               SourceBuffer,           //TermArg=>BufferTerm
                               ByteIndex,              //TermArg=>Integer
                               WordFieldName           //NameString
                           )

DeviceTerm               := Device(
                               DeviceName              //NameString
                           ) {ObjectList}

EventTerm                := Event(
                               EventName               //NameString
                           )

FieldTerm                := Field(
                               RegionName,             //NameString
                               AccessType,            //AccessTypeKeyword
                               LockRule,              //LockRuleKeyword
                               UpdateRule            //UpdateRuleKeyword
                           ) {FieldUnitList}

IndexFieldTerm           := IndexField(
                               IndexName,             //NameString
                               DataName,              //NameString
                               AccessType,            //AccessTypeKeyword
                               LockRule,              //LockRuleKeyword
                               UpdateRule            //UpdateRuleKeyword
                           ) {FieldUnitList}

MethodTerm               := Method(
                               MethodName,            //NameString
                               NumArgs,               //Nothing | ByteConst
                               SerializeRule          //Nothing |
                               //SerializeRuleKeyword
                           ) {TermList}

MutexTerm                := Mutex(
                               MutexName,            //NameString
                               SyncLevel             //ByteConst
                           )

OpRegionTerm             := OperationRegion(
                               RegionName,            //NameString
                               RegionSpace,           //RegionSpaceKeyword
                               Offset,               //TermArg=>DWordConst
                               Length                //TermArg=>DWordConst
                           )

PowerResTerm             := PowerResource(
                               ResourceName,          //NameString
                               SystemLevel,          //ByteConst
                               ResourceOrder         //WordConst
                           ) {ObjectList}

ProcessorTerm            := Processor(
                               ProcessorName,         //NameString
                               ProcessorID,          //ByteConst
                               PBlockAddress,        //DWordConst
                               PblockLength         //ByteConst
                           ) {ObjectList}

ThermalZoneTerm          := ThermalZone(
                               ThermalZoneName       //NameString
                           ) {ObjectList}

AliasTerm                := Alias(
                               SourceObject,          //NameString
                               AliasObject           //NameString
                           )

NameTerm                 := Name(
                               ObjectName,           //NameString
                               Object                //DataObject
                           )

```

```

ScopeTerm          := Scope(
                        Location                //NameString
                        ) {ObjectList}

BreakTerm           := Break

BreakPointTerm      := BreakPoint

FatalTerm           := Fatal(
                        Type,                  //ByteConst
                        Code,                  //DWordConst
                        Arg                    //TermArg=>Integer
                        )

IfElseTerm          := IfTerm ElseTerm

IfTerm              := If(
                        Predicate              //TermArg=>Integer
                        ) {TermList}

ElseTerm            := Nothing | <Else {TermList}>

LoadTerm            := Load(
                        Object,                //NameString
                        DDBHandle             //SuperName
                        )

NoOpTerm            := Noop

NotifyTerm          := Notify(
                        Object,               //SuperName
                        NotificationValue     //TermArg=>ByteConst
                        )

ReleaseTerm         := Release(
                        SyncObject            //SuperName
                        )

ResetTerm           := Reset(
                        SyncObject            //SuperName
                        )

ReturnTerm          := Return(
                        Arg                   //TermArg=>DataObject
                        )

SignalTerm          := Signal(
                        SyncObject            //SuperName
                        )

SleepTerm           := Sleep(
                        MilliSecs             //TermArg=>Integer
                        )

StallTerm           := Stall(
                        MicroSecs             //TermArg=>Integer
                        )

UnloadTerm          := Unload(
                        DDBHandle             //SuperName
                        )

WhileTerm           := While(
                        Predicate              //TermArg=>Integer
                        ) {TermList}

AcquireTerm         := Acquire(
                        SyncObject,           //SuperName
                        TimeoutValue          //WordConst
                        ) => Boolean          //Ones means timed-out

```

AddTerm	<pre> := Add(Addend1, Addend2, Result) => Integer </pre>	<pre> //TermArg=>Integer //TermArg=>Integer //Nothing SuperName </pre>
AndTerm	<pre> := And(Source1, Source2, Result) => Integer </pre>	<pre> //TermArg=>Integer //TermArg=>Integer //Nothing SuperName </pre>
ConcatTerm	<pre> := Concatenate(Source1, Source2, Result) => ComputationalData </pre>	<pre> //TermArg=>ComputationalData //TermArg=>ComputationalData //Nothing SuperName </pre>
CondRefOfTerm	<pre> := CondRefOf(Source, Destination) => Boolean </pre>	<pre> //SuperName //SuperName </pre>
DecTerm	<pre> := Decrement(Addend) => Integer </pre>	<pre> //SuperName </pre>
DerefOfTerm	<pre> := DerefOf(Source) => ObjectReference </pre>	<pre> //TermArg=>ObjectReference //ObjectReference is an object //produced by terms such as //Index, RefOf or CondRefOf. </pre>
DivideTerm	<pre> := Divide(Dividend, Divisor, Remainder, Result) => Integer </pre>	<pre> //TermArg=>Integer //TermArg=>Integer //Nothing SuperName //Nothing SuperName //returns Result </pre>
FindSetLeftBitTerm	<pre> := FindSetLeftBit(Source, Result) => Integer </pre>	<pre> //TermArg=>Integer //Nothing SuperName </pre>
FindSetRightBitTerm	<pre> := FindSetRightBit(Source, Result) => Integer </pre>	<pre> //TermArg=>Integer //Nothing SuperName </pre>
FromBCDTerm	<pre> := FromBCD(BCDValue, Result) => Integer </pre>	<pre> //TermArg=>Integer //Nothing SuperName </pre>
IncTerm	<pre> := Increment(Addend) => Integer </pre>	<pre> //SuperName </pre>
IndexTerm	<pre> := Index(Source, Index, Destination) => ObjectReference </pre>	<pre> //TermArg=> //<BufferTerm PackageTerm> //TermArg=>Integer //Nothing SuperName </pre>
LAndTerm	<pre> := LAnd(Source1, Source2) => Boolean </pre>	<pre> //TermArg=>Integer //TermArg=>Integer </pre>

```

LEqualTerm                := LEqual(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               ) => Boolean

LGreaterTerm               := LGreater(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               ) => Boolean

LGreaterEqualTerm          := LGreaterEqual(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               ) => Boolean

LLessTerm                  := LLess(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               ) => Boolean

LLessEqualTerm             := LLessEqual(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               ) => Boolean

LNotTerm                   := LNot(
                               Source,           //TermArg=>Integer
                               ) => Boolean

LNotEqualTerm              := LNotEqual(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               ) => Boolean

LOrTerm                    := LOr(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               ) => Boolean

MatchTerm                  := Match(
                               SearchPackage,     //TermArg=>Package
                               Op1,               //MatchOpKeyword
                               MatchObject1,     //TermArg=>Integer
                               Op2,               //MatchOpKeyword
                               MatchObject2,     //TermArg=>Integer
                               StartIndex        //TermArg=>Integer
                               ) => Ones | Integer

MultiplyTerm               := Multiply(
                               Multiplicand,      //TermArg=>Integer
                               Multiplier,       //TermArg=>Integer
                               Result            //Nothing | SuperName
                               ) => Integer

NAndTerm                   := NAnd(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               Result            //Nothing | SuperName
                               ) => Integer

NOrTerm                    := NOr(
                               Source1,           //TermArg=>Integer
                               Source2           //TermArg=>Integer
                               Result            //Nothing | SuperName
                               ) => Integer

NotTerm                    := Not(
                               Source,           //TermArg=>Integer
                               Result            //Nothing | SuperName
                               ) => Integer

ObjectTypeTerm             := ObjectType(
                               Object             //SuperName
                               ) => Integer

```



```

OrTerm                := Or(
                        Source1,           //TermArg=>Integer
                        Source2           //TermArg=>Integer
                        Result             //Nothing | SuperName
                        ) => Integer

RefOfTerm              := RefOf(
                        Object             //SuperName
                        ) => ObjectReference

ShiftLeftTerm         := ShiftLeft(
                        Source,           //TermArg=>Integer
                        ShiftCount       //TermArg=>Integer
                        Result            //Nothing | SuperName
                        ) => Integer

ShiftRightTerm        := ShiftRight(
                        Source,           //TermArg=>Integer
                        ShiftCount       //TermArg=>Integer
                        Result            //Nothing | SuperName
                        ) => Integer

SizeOfTerm             := SizeOf(
                        DataObject        //SuperName=>DataObject
                        ) => Integer

StoreTerm              := Store(
                        Source,           //TermArg=>DataObject
                        Destination       //SuperName
                        ) => DataObject

SubtractTerm           := Subtract(
                        Addend1,          //TermArg=>Integer
                        Addend2,          //TermArg=>Integer
                        Result            //Nothing | SuperName
                        ) => Integer

ToBCDTerm              := ToBCD(
                        Value,            //TermArg=>Integer
                        Result            //Nothing | SuperName
                        ) => Integer

WaitTerm               := Wait(
                        SyncObject,       //SuperName
                        TimeoutValue      //TermArg=>Integer
                        ) => Boolean

XOrTerm                := XOr(
                        Source1,          //TermArg=>Integer
                        Source2           //TermArg=>Integer
                        Result            //Nothing | SuperName
                        ) => Integer

ObjectTypeKeyword      := UnknownObj | IntObj | StrObj | BuffObj | PkgObj |
                        FieldUnitObj | DeviceObj | EventObj | MethodObj | MutexObj |
                        OpRegionObj | PowerResObj | ThermalZoneObj | BuffFieldObj |
                        DDBHandleObj

AcessTypeKeyword       := AnyAcc | ByteAcc | WordAcc | DWordAcc | BlockAcc |
                        SMBSendRecvAcc | SMBQuickAcc

LockRuleKeyword        := Lock | NoLock

UpdateRuleKeyword      := Preserve | WriteAsOnes | WriteAsZeros

RegionSpaceKeyword     := UserDefRegionSpace | SystemIO | SystemMemory | PCI_Config |
                        EmbeddedControl | SMBus

UserDefRegionSpace     := 0x80-0xff

SerializeRuleKeyword   := Serialized | NotSerialized

MatchOpKeyword         := MTR | MEQ | MLE | MLT | MGE | MGT

DMATypeKeyword         := Compatibility | TypeA | TypeB | TypeF

```

```

BusMasterKeyword      := BusMaster | NotBusMaster
XferTypeKeyword       := Transfer8 | Transfer16 | Transfer8_16

ResourceTypeKeyword   := ResourceConsumer | ResourceProducer
MinKeyword            := MinFixed | MinNotFixed
MaxKeyword            := MaxFixed | MaxNotFixed
DecodeKeyword         := SubDecode | PosDecode
RangeTypeKeyword      := ISAOnlyRanges | NonISAOnlyRanges | EntireRange
MemTypeKeyword        := Cacheable | WriteCombining | Prefetchable | NonCacheable
ReadWriteKeyword      := ReadWrite | ReadOnly
InterruptTypeKeyword  := Edge | Level
InterruptLevel        := ActiveHigh | ActiveLow
ShareTypeKeyword      := Shared | Exclusive
IODecodeKeyword       := Decode16 | Decode10

SuperName             := NameString | ArgTerm | LocalTerm | DebugTerm | IndexTerm
ArgTerm               := Arg0 | Arg1 | Arg2 | Arg3 | Arg4 | Arg5 | Arg6
LocalTerm             := Local0 | Local1 | Local2 | Local3 | Local4 | Local5 | Local6
                     | Local7
DebugTerm             := Debug

Integer               := ByteConst | WordConst | DWordConst
ByteConst             := 0x00-0xff
WordConst             := 0x0000-0xffff
DWordConst            := 0x00000000-0xffffffff
String               := ''' AsciiCharList '''
AsciiCharList        := Nothing | <AsciiChar AsciiCharList>
AsciiChar             := 0x01-0x7f
NullChar             := 0x00
ConstTerm            := Zero | One | Ones | Revision
Boolean              := True | False
True                  := Ones
False                 := Zero

BufferTerm           := Buffer(
                        BuffSize                //Nothing |
                                                //TermArg=>Integer
                        ) {String | ByteList}

ByteList             := Nothing | <ByteConst ByteListTail>
ByteListTail         := Nothing | <',' ByteConst ByteListTail>

DWordList            := Nothing | <DWordConst DWordListTail>
DWordListTail        := Nothing | <',' DWordConst DWordListTail>

PackageTerm          := Package(
                        NumElements            //Nothing |
                                                //ByteConst
                        ) {PackageList}

PackageList          := Nothing | <PackageElement PackageListTail>
PackageListTail      := Nothing | <',' PackageElement PackageListTail>
PackageElement       := DataObject | NameString

EISAIDTerm           := EISAID(
                        EISAIDString          //String
                        ) => DWordConst

ResourceTemplateTerm  := ResourceTemplate() {ResourceMacroList} => BufferTerm

ResourceMacroList     := Nothing | <ResourceMacroTerm ResourceMacroList>
ResourceMacroTerm     := DMATerm | DWordIOTerm | DWordMemoryTerm | EndDependentFnTerm
                     | FixedIOTerm | InterruptTerm | IOTerm | IRQNoFlagsTerm |
                     | IRQTerm | Memory24Term | Memory32FixedTerm | Memory32Term |
                     | QWordIOTerm | QWordMemoryTerm | StartDependentFnTerm |
                     | StartDependentFnNoPriTerm | VendorLongTerm | VendorShortTerm
                     | WordBusNumberTerm | WordIOTerm

DMATerm              := DMA(
                        DMAType,                //DMATypeKeyword (_TYP)
                        BusMaster,              //BusMasterKeyword (_BM)
                        XferType,                //XferTypeKeyword (_SIZ)
                        ResourceTag              //Nothing | NameString
                        ) {ByteList}            //List of channels (0-17)

```

```

DWordIOTerm      := DWORDIO(
    ResourceType,           //Nothing (ResourceConsumer) |
                             //ResourceTypeKeyword
    MinType,               //Nothing (MinNotFixed) |
                             //MinKeyword (_MIF)
    MaxType,               //Nothing (MaxNotFixed) |
                             //MaxKeyword (_MAF)
    Decode,                //Nothing (PosDecode) |
                             //DecodeKeyword (_DEC)
    RangeType,             //Nothing (EntireRange) |
                             //RangeTypeKeyword (_RNG)
    AddressGranularity,    //DWordConst (_GRA)
    MinAddress,            //DWordConst (_MIN)
    MaxAddress,            //DWordConst (_MAX)
    Translation,           //DWordConst (_TRA)
    AddressLen,            //DWordConst (_LEN)
    ResSourceIndex,        //Nothing | ByteConst
    ResSource,             //Nothing | String
    ResourceTag            //Nothing | NameString
)

DWordMemoryTerm   := DWORDMemory(
    ResourceType,           //Nothing (ResourceConsumer) |
                             //ResourceTypeKeyword
    Decode,                //Nothing (PosDecode) |
                             //DecodeKeyword (_DEC)
    MinType,               //Nothing (MinNotFixed) |
                             //MinKeyword (_MIF)
    MaxType,               //Nothing (MaxNotFixed) |
                             //MaxKeyword (_MAF)
    MemType,               //Nothing (NonCacheable) |
                             //MemTypeKeyword (_MEM)
    ReadWriteType,         //ReadWriteKeyword (_RW)
    AddressGranularity,    //DWordConst (_GRA)
    MinAddress,            //DWordConst (_MIN)
    MaxAddress,            //DWordConst (_MAX)
    Translation,           //DWordConst (_TRA)
    AddressLen,            //DWordConst (_LEN)
    ResSourceIndex,        //Nothing | ByteConst
    ResSource,             //Nothing | String
    ResourceTag            //Nothing | NameString
)

EndDependentFnTerm := EndDependentFn()

FixedIOTerm       := FixedIO(
    AddressBase,           //WordConst (_BAS)
    RangeLen,              //ByteConst (_LEN)
    ResourceTag            //Nothing | NameString
)

InterruptTerm     := Interrupt(
    ResourceType,           //Nothing (ResourceConsumer) |
                             //ResourceTypeKeyword
    InterruptType,         //InterruptTypeKeyword
                             //( _LL, _HE)
    InterruptLevel,        //InterruptLevelKeyword
                             //( _LL, _HE)
    ShareType,             //Nothing (Exclusive)
                             //ShareTypeKeyword (_SHR)
    ResSourceIndex,        //Nothing | ByteConst
    ResSource,             //Nothing | String
    ResourceTag            //Nothing | NameString
) {DWordList}           //list of interrupts (_INT)

IOTerm            := IO(
    IODecode,              //IODecodeKeyword (_DEC)
    MinAddress,            //WordConst (_MIN)
    MaxAddress,            //WordConst (_MAX)
    Alignment,             //ByteConst (_ALN)
    RangeLen,              //ByteConst (_LEN)
    ResourceTag            //Nothing | NameString
)

```

```

IRQNoFlagsTerm      := IRQNoFlags(
                        ResourceTag           //Nothing | NameString
                        ) {ByteList}         //list of interrupts (0-15)

IRQTerm             := IRQ(
                        InterruptType,       //InterruptTypeKeyword
                                           //( _LL, _HE)
                        InterruptLevel,      //InterruptLevelKeyword
                                           //( _LL, _HE)
                        ShareType,          //Nothing (Exclusive)
                                           //ShareTypeKeyword ( _SHR)
                        ResourceTag         //Nothing | NameString
                        ) {ByteList}         //list of interrupts (0-15)

Memory24Term        := Memory24(
                        ReadWriteType,      //ReadWriteKeyword ( _RW)
                        MinAddress[23:8],   //WordConst ( _MIN)
                        MaxAddress[23:8],   //WordConst ( _MAX)
                        Alignment,          //WordConst ( _ALN)
                        RangeLen,           //WordConst ( _LEN)
                        ResourceTag         //Nothing | NameString
                        )

Memory32FixedTerm   := Memory32Fixed(
                        ReadWriteType,      //ReadWriteKeyword ( _RW)
                        AddressBase,        //DWordConst ( _BAS)
                        RangeLen,           //DWordConst ( _LEN)
                        ResourceTag         //Nothing | NameString
                        )

Memory32Term        := Memory32(
                        ReadWriteType,      //ReadWriteKeyword ( _RW)
                        MinAddress,         //DWordConst ( _MIN)
                        MaxAddress,         //DWordConst ( _MAX)
                        Alignment,          //DWordConst ( _ALN)
                        RangeLen,           //DWordConst ( _LEN)
                        ResourceTag         //Nothing | NameString
                        )

QWordIOTerm         := QWORDIO(
                        ResourceType,       //Nothing (ResourceConsumer) |
                                           //ResourceTypeKeyword
                        MinType,            //Nothing (MinNotFixed) |
                                           //MinKeyword ( _MIF)
                        MaxType,            //Nothing (MaxNotFixed) |
                                           //MaxKeyword ( _MAF)
                        Decode,             //Nothing (PosDecode) |
                                           //DecodeKeyword ( _DEC)
                        RangeType,          //Nothing (EntireRange) |
                                           //RangeTypeKeyword ( _RNG)
                        AddressGranularity, //QWordConst ( _GRA)
                        MinAddress,         //QWordConst ( _MIN)
                        MaxAddress,         //QWordConst ( _MAX)
                        Translation,        //QWordConst ( _TRA)
                        AddressLen,         //QWordConst ( _LEN)
                        ResSourceIndex,     //Nothing | ByteConst
                        ResSource,          //Nothing | String
                        ResourceTag         //Nothing | NameString
                        )

```

```

QWordMemoryTerm      := QWORDMemory(
    ResourceType,      //Nothing (ResourceConsumer) |
                        //ResourceTypeKeyword
    Decode,             //Nothing (PosDecode) |
                        //DecodeKeyword (_DEC)
    MinType,            //Nothing (MinNotFixed) |
                        //MinKeyword (_MIF)
    MaxType,            //Nothing (MaxNotFixed) |
                        //MaxKeyword (_MAF)
    MemType,            //Nothing (NonCacheable) |
                        //MemTypeKeyword (_MEM)
    ReadWriteType,      //ReadWriteKeyword (_RW)
    AddressGranularity, //QWordConst (_GRA)
    MinAddress,          //QWordConst (_MIN)
    MaxAddress,          //QWordConst (_MAX)
    Translation,        //QWordConst (_TRA)
    AddressLen,          //QWordConst (_LEN)
    ResSourceIndex,      //Nothing | ByteConst
    ResSource,           //Nothing | String
    ResourceTag          //Nothing | NameString
)

StartDependentFnTerm := StartDependentFn(
    CompatPriority,      //ByteConst (0-2)
    PerfRobustPriority   //ByteConst (0-2)
) {ResourceMacroList}

StartDependentFnNoPriTerm := StartDependentFnNoPri() {ResourceMacroList}

VendorLongTerm := VendorLong(
    ResourceTag          //Nothing | NameString
) {ByteList}

VendorShortTerm := VendorShort(
    ResourceTag          //Nothing | NameString
) {ByteList} //up to 7 bytes

WordBusNumberTerm := WordBusNumber(
    ResourceType,      //Nothing (ResourceConsumer) |
                        //ResourceTypeKeyword
    MinType,            //Nothing (MinNotFixed) |
                        //MinKeyword (_MIF)
    MaxType,            //Nothing (MaxNotFixed) |
                        //MaxKeyword (_MAF)
    Decode,             //Nothing (PosDecode) |
                        //DecodeKeyword (_DEC)
    AddressGranularity, //WordConst (_GRA)
    MinAddress,          //WordConst (_MIN)
    MaxAddress,          //WordConst (_MAX)
    Translation,        //WordConst (_TRA)
    AddressLen,          //WordConst (_LEN)
    ResSourceIndex,      //Nothing | ByteConst
    ResSource,           //Nothing | String
    ResourceTag          //Nothing | NameString
)

```

```

WordIOTerm                               := WordIO(
    ResourceType,                        //Nothing (ResourceConsumer) |
                                         //ResourceTypeKeyword
    MinType,                             //Nothing (MinNotFixed) |
                                         //MinKeyword (_MIF)
    MaxType,                             //Nothing (MaxNotFixed) |
                                         //MaxKeyword (_MAF)
    Decode,                              //Nothing (PosDecode) |
                                         //DecodeKeyword (_DEC)
    RangeType,                           //Nothing (EntireRange) |
                                         //RangeTypeKeyword (_RNG)
    AddressGranularity,                  //WordConst (_GRA)
    MinAddress,                          //WordConst (_MIN)
    MaxAddress,                          //WordConst (_MAX)
    Translation,                         //WordConst (_TRA)
    AddressLen,                          //WordConst (_LEN)
    ResSourceIndex,                      //Nothing | ByteConst
    ResSource,                           //Nothing | String
    ResourceTag                           //Nothing | NameString
)

```

15.2 Full ASL Reference

This reference section is for developers who are writing ASL code while developing definition blocks for platforms.

15.2.1 ASL Names

This section describes how to encode object names using ASL.

The following table lists the characters legal in any position in an ASL object name.

Table 15-2 Control Method Named Object Reference Encodings

Value	Description	
41-5A, 5F	Lead character of name ('A' - 'Z', ' _')	LeadNameChar
30-39, 41-5A, 5F	Non-lead (trailing) character of name ('A' - 'Z', ' _', '0 - 9')	NameChar

The following table lists the name modifiers.

Table 15-3 Definition Block Name Modifier Encodings

	Description	NamePrefix :=	Followed by ...
5C	Name space root ('\')	RootPrefix	Name
5E	Parent name space ('^')	ParentPrefix	Name
2E	Name extender: 1	DualNamePrefix	Name Name
2F	Name extender: N	MultiNamePrefix	count Name ^{count}

15.2.2 ASL Data Types

The contents of an object, or the data it references, may be abstract entities (for example, “Device Object”) or can be one of three computational data types. The computational data type can be used as arguments to many of the ASL Operator terms.

Table 15-4 Data Types

Data Type	Description
Integer	32-bit little endian unsigned value.
Buffer	Arbitrary fixed length array of bytes.
String	ASCIIZ string 1 to 200 characters in length (including NullChar).

15.2.3 ASL Terms

This section describes all the ASL terms and provides sample ASL code that uses the terms.

The ASL terms are grouped into the following categories:

- Definition block term
- Compiler directive terms
- Object terms
- Opcode terms
- User terms
- Data objects
- Miscellaneous objects

15.2.3.1 Definition Block Term

```
DefinitionBlockTerm      := DefinitionBlock(
                           AMLFileName,           //String
                           TableSignature,        //String
                           ComplianceRevision,    //ByteConst
                           OEMID,                //String
                           TableID,              //String
                           OEMRevision           //DWordConst
                           ) {TermList}
```

The **DefinitionBlock** term specifies the unit of data and/or AML code that the OS will load as part of the Differentiated Definition Block or as part of an additional Definition Block. This unit of data and/or AML code describes either the base system or some large extension (such as a docking station). The entire DefinitionBlock will be loaded and compiled by the OS as a single unit, and can be unloaded by the OS as a single unit.

15.2.3.2 Compiler Directive Terms

The compiler directives are:

- Include term
- External term

15.2.3.2.1 Include – Include Another ASL File

```
IncludeTerm              := Include(
                           IncFilePathName        //String
                           )
```

IncFilePathname is the full OS file system path to another file that contains ASL terms to be included in the current file of ASL terms.

15.2.3.2.2 External – Declare External Objects

```
ExternalTerm             := External(
                           ObjName,              //NameString
                           ObjType              //Nothing | ObjectTypeKeyword
                           )
```

The External compiler directive is to let the assembler know that the object is declared external to this table so that the assembler will not complain about the undeclared object. During compiling, the assembler will create the external object at the specified place in the name space (if a full path of the object is specified), or the object will be created at the current scope of the External term. *ObjType* is optional. If not specified, "UnknownObj" type is assumed.

15.2.3.3 Object Terms

Object terms includes: Named Object terms and Name Space Modifiers.

15.2.3.3.1 Named Object Terms

The ASL terms that can be used to create named objects in a definition block are listed in the following table.

Table 15-5 Named Object Terms

ASL Statement	Description
BankField	Declares fields in a banked configuration object.
CreateBitField	Declare a bit field object of a buffer object.
CreateByteField	Declare a byte field object of a buffer object.
CreateDWordField	Declare a dword field object of a buffer object.
CreateField	Declare a field object of any bit length of a buffer object.
CreateWordField	Declare a dword field object of a buffer object.
Device	Declares a bus/device object.
Event	Declares an event synchronization object.
Field	Declares fields of an operation region object.
IndexField	Declares fields in an index/data configuration object.
Method	Declares a control method.
Mutex	Declares a mutex synchronization object.
OperationRegion	Declares an operational region.
PowerResource	Declares a power resource object.
Processor	Declares a processor package.
ThermalZone	Declares a thermal zone package.

15.2.3.3.1.1 BankField - Declare Bank/Data Field

```
BankFieldTerm      := BankField(
                        RegionName,           //NameString
                        BankName,             //NameString
                        BankValue,            //TermArg=>DWordConst
                        AccessType,           //AccessTypeKeyword
                        LockRule,             //LockRuleKeyword
                        UpdateRule,          //UpdateRuleKeyword
                    ) {FieldUnitList}
```

This statement creates data field objects. The contents of the created objects are obtained by a reference to a bank selection register.

This encoding is used to define named data field objects whose data values are fields within a larger object selected by a bank selected register. Accessing the contents of a banked field data object will occur automatically through the proper bank setting, with synchronization occurring on the operation region that contains the *BankName* data variable, and on the global lock if specified by the *LockRule*.

The *AccessType*, *LockRule*, *UpdateRule*, and *FieldUnitList* are the same format as the *Field* operator.

The following is a block of ASL sample code using **BankField**:

- Creates a 4-bit bank select register in system I/O space.
- Creates overlapping fields in the same system I/O space which are selected via the bank register.


```

// define 256-byte operational region in SystemIO space
// and name it GIO0
OperationRegion (GIO0, SystemIO, 0x125, 0x100)

// create some field in GIO including a 4 bit bank select register
Field (GIO0, ByteAcc, NoLock, Preserve) {
    GLB1, 1,
    GLB2, 1,
    Offset(1),          // move to offset for byte 1
    BNK1, 4
}

// Create FET0 & FET1 in bank 0 at byte offset 0x30
BankField (GIO0, BNK1, 0, ByteAcc, NoLock, Preserve) {
    Offset (0x30),
    FET0, 1,
    FET1, 1
}

// Create BLVL & BAC in bank 1 at the same offset
BankField (GIO0, BNK1, 1, ByteAcc, NoLock, Preserve) {
    Offset (0x30),
    BLVL, 7,
    BAC, 1
}

```

15.2.3.3.1.2 CreateBitField

```

CreateBitFieldTerm      := CreateBitField(
                           SourceBuffer,          //TermArg=>BufferTerm
                           BitIndex,              //TermArg=>Integer
                           BitFieldName           //NameString
                           )

```

SourceBuffer is evaluated as a buffer. *BitIndex* is evaluated as an integer. A new buffer field object *BitFieldName* is created for the bit of *SourceBuffer* at the bit index of *BitIndex*. The bit-defined field within *SourceBuffer* must exist.

15.2.3.3.1.3 CreateByteField

```

CreateByteFieldTerm     := CreateByteField(
                           SourceBuffer,          //TermArg=>BufferTerm
                           ByteIndex,            //TermArg=>Integer
                           ByteFieldName         //NameString
                           )

```

SourceBuffer is evaluated as a buffer. *ByteIndex* is evaluated as an integer. A new buffer field object *ByteFieldName* is created for the byte of *SourceBuffer* at the byte index of *ByteIndex*. The byte-defined field within *SourceBuffer* must exist.

15.2.3.3.1.4 CreateDWordField

```

CreateDWordFieldTerm    := CreateDWordField(
                           SourceBuffer,          //TermArg=>BufferTerm
                           ByteIndex,            //TermArg=>Integer
                           DWordFieldName        //NameString
                           )

```

SourceBuffer is evaluated as a buffer. *ByteIndex* is evaluated as an integer. A new buffer field object *DWordFieldName* is created for the DWord of *SourceBuffer* at the byte index of *ByteIndex*. The DWord-defined field within *SourceBuffer* must exist.

15.2.3.3.1.5 CreateField - Field

```

CreateFieldTerm         := CreateField(
                           SourceBuffer,          //TermArg=>BufferTerm
                           BitIndex,            //TermArg=>Integer
                           NumBits,             //TermArg=>Integer
                           FieldName            //NameString
                           )

```

SourceBuffer is evaluated as a buffer. *BitIndex* and *NumBits* are evaluated as integers. A new buffer field object *FieldName* is created for the bits of *SourceBuffer* at *BitIndex* for *NumBits*. The entire bit range of the defined field within *SourceBuffer* must exist.

15.2.3.3.1.6 CreateWordField

```
CreateWordFieldTerm      := CreateWordField(
                               SourceBuffer,      //TermArg=>BufferTerm
                               ByteIndex,          //TermArg=>Integer
                               WordFieldName        //NameString
                           )
```

SourceBuffer is evaluated as a buffer. *ByteIndex* is evaluated as an integer. A new bufferfield object *WordFieldName* is created for the word of *SourceBuffer* at the word index of *ByteIndex*. The word-defined field within *SourceBuffer* must exist.

15.2.3.3.1.7 Device - Declare Bus/Device Package

```
DeviceTerm               := Device(
                               DeviceName          //NameString
                           ) {ObjectList}
```

Creates a Device object, which represents either a bus or a device or any other such entity of use. **Device** opens a name scope.

A Bus/Device Package is one of the basic ways the Differentiated Definition Block describes the hardware devices in the system to the operating software. Each Bus/Device Package is defined somewhere in the hierarchical name space corresponding to that device's location in the system. Within the name space of the device are other names that provide information and control of the device, along with any sub-devices that in turn describe sub-devices, and so on.

For any device, the BIOS provides only information that is added to the device in a non-hardware standard manner. This type of "value added" function is expressible in the ACPI Definition Block such that operating software can use the function.

The BIOS supplies Device Objects only for devices that are obtaining some system-added function outside the device's normal capabilities and for any Device Object required to fill in the tree for such a device. For example, if the system includes a PCI device (integrated or otherwise) with no additional functions such as power management, the BIOS would not report such a device; however, if the system included an integrated ISA device below the integrated PCI device (device is an ISA bridge), then the system would include a Device Package for the ISA device with the minimum feature being added being the ISA device's ID and configuration information and the parent PCI device, because it is required to get the ISA Device Package placement in the Name Space correct.

The following block of ASL sample code shows a nested use of Device objects to describe an IDE controller connected to the root PCI bus.

```

Device (IDE0) {          // primary controller
    Name(_ADR, 0)        // put PCI Address (device/function) here

    // define region for IDE mode register
    OperationRegion (PCIC, PCI_Config, 0x50, 0x10)
    Field (PCIC, AnyAcc, NoLock, Preserve) {
        ...
    }
    Device (PRIM) {      //Primary adapter
        Name(_ADR, 0)    //Primary adapter = 0
        ...

        Device (MSTR) { // master channel
            Name(_ADR, 0)
            Name(_PR0, Package() {0, PIDE})
            Method (_STM, 2) {
                ...
            }
        }
    }

    Device (SLAV) {
        Name(_ADR, 1)
        Name(_PR0, Package() {0, PIDE})
        Method (_STM, 2) {
            ...
        }
    }
}
}
}

```

15.2.3.3.1.8 Event - Declare Event Synchronization Object

```

EventTerm                := Event(
                                EventName                //NameString
                            )

```

Creates an event synchronization object named *EventName*.

For more information about the uses of an event synchronization object, see the ASL definitions for the Wait, Signal, and Reset function operators.

15.2.3.3.1.9 Field - Declare Field Objects

```

FieldTerm                := Field(
                                RegionName,                //NameString
                                AccessType,                //AccessTypeKeyword
                                LockRule,                  //LockRuleKeyword
                                UpdateRule,                //UpdateRuleKeyword
                            ) {FieldUnitList}

```

Declares a series of named data objects whose data values are fields within a larger object. The fields are parts of the object named by *RegionName*, but their names appear in the same scope as the **Field** term.

For example, the field operator allows a larger operation region that represents a hardware register to be broken down into individual bit fields that can then be accessed by the bit field names. Extracting and combining the component field from its parent is done automatically when the field is accessed.

Accessing the contents of a field data object provides access to the corresponding field within the parent object. If the parent object supports Mutex synchronization, accesses to modify the component data objects will acquire and release ownership of the parent object around the modification.

All accesses within the parent object are performed naturally aligned. If desired, *AccessType* can be used to force minimum access width. Note that the parent object must be able to accommodate the *AccessType* width. For example, an access type of **WordAcc** cannot read the last byte of an odd-length operation region. Not all access types are meaningful for every type of operational region.

The following table relates region types declared with an **OperationRegion** term to the different access types supported for each region.

Table 15-6 OperationRegion Region Types and Access Types

Region Types	Access Type	Description
SystemMemory SystemIO PCI_Config	ByteAcc WordAcc DWordAcc AnyAcc	Read/Write Byte, Word, DWord access
EmbeddedControl	ByteAcc	
SMBus	ByteAcc WordAcc BlockAcc AnyAcc SMBSendRecvAcc SMBQuickAcc	Read/Write SMBus byte protocol Read/Write SMBus word protocol Read/Write SMBus block protocol Read/Write linear SMBus byte, word, block protocol Send/Receive SMBus protocol QuickRead/QuickWrite SMBus protocol

If *LockRule* is set to **Lock**, accesses to modify the component data objects will acquire and release the global lock. If both types of locking occur, the global lock is acquired after the parent object Mutex.

UpdateRule is used to specify how the unmodified bits of a field are treated. For example, if a field defines a component data object of 4 bits in the middle of a **WordAcc** region, when those 4 bits are modified the *UpdateRule* specifies how the other 12 bits are treated.

The named data objects are provided in *FieldList* as a series of names and bit widths. Bits assigned no name (or NULL) are skipped. The ASL compiler supports an **Offset(ByteOffset)** macro within a *FieldList* to skip to the bit position of the supplied byte offset.

For support of non-linear address devices, such as SMBus devices, a protocol is required to be associated with each command value. The ASL compiler supports the **AccessAs(AccessType, AccessAttribute)** macro within a *FieldList*. The *AccessAttribute* portion of the macro is interpreted differently depending on the address space. For **SystemMemory**, **SystemIO**, **PCI_Config** or **EmbeddedControl** space the *AccessAttribute* is reserved. For **SMBus** devices the *AccessAttribute* indicates the command value of the SMBus device to use for the field being defined. The *AccessAttribute* allows a specific protocol to be associated with the fields following the macro and can contain any of the Access Type listed in the table.

15.2.3.3.1.9.1 SMBus Slave Address

SMBus device Addressing supports both a linear and non-linear addressing mechanism. This section clarifies how ACPI treats these types of devices and how they should be defined and accessed. SMBus devices are defined to have a fixed 7-bit slave address. This can be illustrated by the smart battery subsystem devices:

Table 15-7 Examples of SMBus Devices and Slave Addresses

SMBus Device Description	Slave Address (A0-A6)
SMBus Host Slave Interface	0x8
SBS Charger	0x9
SBS Selector	0xA
SBS Battery	0xB

The SMBus driver expects a 7-bit slave address for the device to be passed to it. The 1.0 System Management Bus specification defines the address protocols (how data is passed on the wiggling pins) as:

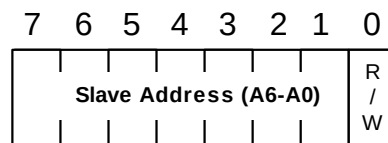


Figure 15-1 SMBus Slave Address Protocol

This indicates that bit 0 of the protocol represents whether this access is a read or write cycle, and the next six bits represent the slave address. Note that the driver expects a zero-based address, not a one-based address. For example, the SBS battery has a slave address of 0xB, or 0001011b (bits 0, 1 and 4 being set). This value is represented by 0x16 for writes or 0x17 for reads to the smart battery in the SMBus protocol format. The protocol format of the slave address and the actual slave address should not be confused as the SMBus driver expects the actual slave address, not the protocol format with the read/write value; the driver will shift the slave address left by 1 bit and mask in the read/write protocol.

15.2.3.3.1.9.2 SMBus Addressing

Associated with each SMBus device is an 8-bit command register that represents an additional address space within the device, allowing up to 256 registers within an SMBus device. For some devices this is treated as a linear address space; for other devices such as the Smart Battery, this is treated as a non-linear address space. The SMBus driver differentiates these types of devices so that it can understand how to use the different SMBus protocols on the device.

A linear address device treats the command and slave address fields as a byte-linear 15-bit address space where the address is formed as follows:

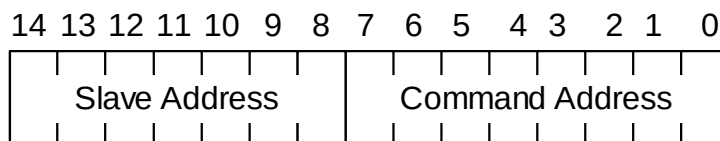


Figure 15-2 SMBus Linear Address Decode

For example an SMBus memory device that consumes slave address 0x40 would be accessing a linear address range of 0x4000-0x40FF (256 bytes of address space). A byte access to 0x4000 (slave 0x40, command 0) would access byte location 0x4000 (slave 0x40, command 0), and a word access to 0x4000 (slave 0x40, command 0) would access byte locations 0x4000-0x4001 (slave 0x40, commands 0-1). For a device that behaves in this manner, ASL should indicate an AnyAcc in the field operator defining the SMBus device. This indicates to the SMBus driver that it can use the read/write block, read/write word, or read/write byte protocols to access this device.

A non-linear address device (such as the smart battery) defines each command value within the device to be a potentially different size. The ACPI driver treats such a device differently from a linear address device by only accessing command values with the specified protocol only. For example the smart battery device has a slave address of 0xB and a definition for the first two command values as follows:

Table 15-8 Example Command Codes from the Smart Battery

Command Address	Data Type	Protocol to Access
0x0	Manufacture Access	Word Read/Write
0x1	Remaining Capacity Alarm	Word Read/Write
0x2	Remaining Time Alarm	Word Read/Write
...		
0x20	Manufacture Name	Block Read/Write
0x21	Device Name	Block Read/Write

The Smart Battery uses a non-linear programming model. Each command register can be a different size and has a specific SMBus protocol associated with it. For example command register 0x0 contains a word of data (which in a linear device would take up two command registers 0 and 1) that represents the “Manufacture Access” and command register 0x1 contains the next word of data (which in a linear device would take up two command registers 0 and 1) that represents the “Remaining Capacity.” In a linear address model these registers would overlap; however, this is legitimate SMBus device definition. As a further example command

register 0x20 can represent up to 32 bytes of data (block read/write) and command register 0x21 also represents up to 32 bytes of data.

15.2.3.3.1.9.3 SMBus Protocols

This section describes the different SMBus protocols and how the SMBus driver treats them. It also gives examples of how to define and then access such devices in ASL.

15.2.3.3.1.9.3.1 Quick Protocol (QuickAcc)

The SMBus Quick protocol does not transfer any data. This protocol is used to control simple devices and consists of the slave address with the R/W bit set high or low. Therefore, two types of Quick commands can be generated: QuickRead with the R/W protocol bit reset LOW or QuickWrite with the R/W protocol bit set HIGH. A device defined to use the quick protocol has no command registers, and consumes the entire 7-bit slave address.

To define a quick device an operation region is generated using the SMBus address type. Next a field is generated in the operation region using the “QuickAcc” access type. To generate a QuickWrite protocol to this device, ASL would generate a write to this field. To generate a QuickRead protocol to this device, ASL would generate a read to this field. Note that even though the ASL read the field and a QuickRead protocol was sent to the device, the device does not return any data and the numeric result returned by the SMB driver to the ASL will be 0. For example,

```
Device(\_SB.EC0) {
    Name(_HID, EISAID("PNP0C09"))
    Name(_CRS,
        ResourceTemplate() {
            IO(Decode16, 0x62, 0x62, 0, 1),
            IO(Decode16, 0x66, 0x66, 0, 1)
        }
    )
    Name(_GPE, Zero) //EC is wired to bit 0 of GPE
    Device (SMB1) {
        Name(_ADR, "ACPI0001")
        Name(_EC, 0x8030) // EC offset(0x80), Query (0x30)
        OperationRegion(Ph01, SMBus, 0x61, 0x1)
        Device(DEVA) {
            Name(_ADR, 0x61) // Slave Address 0x61
            Field(Ph01, QuickAcc, NoLock, Preserve) {
                QCKA, 1
            }
        } // end of DEVA
    } // end of SMB1
} // end of EC0
```

This example creates a quick SMBus device residing at slave address 0x61 called “QCKA”. Examples of generating the Quick0 and Quick1 commands from ASL is illustrated below:

```
Method(Test) {
    Store(1, QCKA) // Generates a QuickRead command to slave address 0x61
    Store(QCKA, Local0) // Generates a QuickWrite command to slave address 0x61
}
```

15.2.3.3.1.9.3.2 Send/Receive Command Protocol (SMBSendRecvAcc)

The SMBus Send/Receive protocol transfers a byte of data between the selected SMBus slave address and the ASL code performing a read/write to the field. The SMBus protocol for send-command is defined that the byte being written is presented in the “command” field, while the data returned from a read-command is defined to be the byte in the data field. The SMBus driver will read and write the data to a SMBSendRecvAcc field accordingly.

To define a send/receive command to a device an operation region is generated using the SMBus address type. Next a field is generated in the operation region using the “SMBSendRecvAcc” access type. To generate a

send byte protocol to this device, ASL would generate a write to this field. To generate a receive byte protocol to this device, ASL would generate a read to this field. For example,

```
Device(\_SB.EC0) {
    Name(_HID, EISAID("PNP0C09"))
    Name(_CRS,
        ResourceTemplate() {
            IO(Decode16, 0x62, 0x62, 0, 1),
            IO(Decode16, 0x66, 0x66, 0, 1)
        }
    )
    Name(_GPE, Zero) //EC is wired to bit 0 of GPE
    Device (SMB1) {
        Name(_ADR, "ACPI0001")
        Name(_EC, 0x8030) // EC offset(0x80), Query (0x30)
        OperationRegion(PH01, SMBus, 0x62, 0x1)
        Device(DEVB) {
            Name(_ADR, 0x62) // Slave Address 0x62
            Field(PH01, SMBSendRecvAcc, NoLock, Preserve) {
                TSTA, 1,
                TSTB, 1,
                TSTC, 5
            }
        } // end of DEVB
    } // end of SMB1
} // end of EC0
```

This example creates a send/receive byte SMBus device residing at slave address 0x62. There are three fields that reference this single byte called “TSTA”, “TSTB” and “TSTC”. Examples of generating the send/receive byte protocols from ASL are illustrated below:

```
Method(Test) {
    Store(1, TSTA) // Sets TSTA, preserved TSTB and TSTC, sendbyte
    Store(0, TSTB) // Clears TSTB, preserved TSTA and TSTC, sendbyte
    Store(0x7, TSTC) // Sets TSTC to 0111b, preserved TSTA and TSTB, sendbyte
    Store(TSTA, Local0) // returns 1, receive byte
    Store(TSTB, Local0) // returns 0, receive byte
    Store(TSTC, Local0) // returns 7, receive byte
}
```

Read/Write Byte Protocol (ByteAcc)

The SMBus Read/Write Byte protocol transfers a byte of data between the selected SMBus slave address and command value. The command address is defined through the use of the `AccessAs(AccessType, AccessAttribute)` macro. In this case the `AccessAttribute` represents the byte aligned command value, and `AccessType` would be set to `ByteAcc`.

To define a `ByteAcc` device an operation region is generated using the SMBus address type. Next a field is generated in the operation region using the “ByteAcc” access type. In the field list an `AccessAs(ByteAcc, command_value)` macro is used to define what command address is associated with this field. The absence of the macro assume a starting command value of 0. The SMBus driver assumes that after the `AccessAs(ByteAcc, command_value)` macro is declared, the next 8-bits represent this command register. If a field is defined that crosses over this 8-bit boundary, then the SMBus driver assumes this field resides in multiple byte-wide command registers with a command address value of `command_value+1` (for each new register) using the `ByteAcc` protocol.

To generate a write byte protocol to this device, ASL would generate a write to this field. To generate a read byte protocol to this device, ASL would generate a read to this field. For example,

```

Device(\_SB.EC0) {
    Name(_HID, EISAID("PNP0C09"))
    Name(_CRS,
        ResourceTemplate() {
            IO(Decode16, 0x62, 0x62, 0, 1),
            IO(Decode16, 0x66, 0x66, 0, 1)
        }
    )
    Name(_GPE, Zero) //EC is wired to bit 0 of GPE
    Device (SMB1) {
        Name(_ADR, "ACPI0001")
        Name(_EC, 0x8030) // EC offset(0x80), Query (0x30)
        OperationRegion(Ph01, SMBus, 0x63, 0x1)
        Device(DEVB) {
            Name(_ADR, 0x63) // Slave Address 0x63
            Field(Ph01, ByteAcc, NoLock, Preserve) {
                AccessAs(ByteAcc, 0),
                TSTA, 1,
                TSTB, 1,
                TSTC, 5,
                TSTD, 4 // this field spans command address 0 and 1
            }
        } // end of DEVB
    } // end of SMB1
} // end of EC0

```

This example creates a read/write byte SMBus device residing at slave address 0x63. There are four fields that use two command registers (0 and 1), called “TSTA”, “TSTB”, “TSTC”, and “TSTD”. TSTA, TSTB and TSTC reference command register 0. TSTD references both command registers 0 and 1: bit0 of TSTD represents bit 7 of command register 0, while bits 1-3 of field TSTD represent bits 0-2 of command register 1. Examples of generating the read/write byte protocols from ASL is illustrated below:

```

Method(Test) {
    Store(1, TSTA) // Sets TSTA, preserved TSTB and TSTC, write byte
    Store(0, TSTB) // Clears TSTB, preserved TSTA and TSTC, write byte
    Store(0x7, TSTC) // Sets TSTC to 0111b, preserved TSTA and TSTB, write byte
    Store(0xF, TSTD) // Sets TSTD to 0xF, command registers 0 and 1
    Store(TSTA, Local0) // returns 1, read byte
    Store(TSTB, Local0) // returns 0, read byte
    Store(TSTC, Local0) // returns 7, read byte
    Store(TSTD, Local0) // returns 0xF from command registers 0 and 1
}

```

15.2.3.3.1.9.3.3 Read/Write Word Protocol (WordAcc)

The SMBus Read/Write Word protocol transfers a word of data between the selected SMBus slave address and command value. The command address is defined through the use of the `AccessAs(AccessType, AccessAttribute)` macro. In this case the `AccessAttribute` represents the byte aligned command value, and `AccessType` should be set to `WordAcc`.

To define a `WordAcc` device an operation region is generated using the SMBus address type. Next a field is generated in the operation region using the “`WordAcc`” access type. In the field list an `AccessAs(WordAcc, command_value)` macro is used to define what command address is associated with this field. The absence of the macro assume a starting command value of 0. The SMBus driver assumes that after the `AccessAs(WordAcc, command_value)` macro is declared, the next 16-bits represent this command register. If a field is defined that crosses over this 16-bit boundary, then the SMBus driver assumes this field resides in multiple word wide command registers with a command address value of `command_value+2` (for each new register) using the `WordAcc` protocol.

To generate a write word protocol to this device, ASL would generate a write to this field. To generate a read word protocol to this device, ASL would generate a read to this field.

15.2.3.3.1.9.3.4 Read/Write Block Protocol (*BlockAcc*)

The SMBus Read/Write Block protocol transfers up to a 32 byte buffer of data between the selected SMBus slave address and command value. The command address is defined through the use of the *AccessAs*(*AccessType*, *AccessAttribute*) macro. In this case the *AccessAttribute* represents the byte aligned command value, and *AccessType* would be set to *BlockAcc*.

To define a *BlockAcc* device an operation region is generated using the SMBus address type. Next a field is generated in the operation region using the “*BlockAcc*” access type. In the field list an *AccessAs*(*BlockAcc*, *command_value*) macro is used to define what command address is associated with this field. The absence of the macro assume a starting command value of 0. The SMBus driver assumes that after the *AccessAs*(*BlockAcc*, *command_value*) macro is declared the command register is 32 bytes or less. Each block field must start on the a *command_value* boundary.

The SMBus driver passes block data to and from ASL through the buffer data type. The buffer is structured such that the byte count of the data to write is in record 0 followed by the buffer data. For example a 5 byte buffer with the contents of 1, 2, 3, 4 would be generated as:

```
Buffer(5){4, 1, 2, 3, 4}
```

Where the length of the buffer is its byte data width plus 1, and the first entry is the length of data (buffer length minus 1). On reads, ASL will return a buffer with the first entry set to the number of data bytes returned. For example,

```
Device(\_SB.EC0) {
    Name(_HID, EISAID("PNP0C09"))
    Name(_CRS,
        ResourceTemplate() {
            IO(Decode16, 0x62, 0x62, 0, 1),
            IO(Decode16, 0x66, 0x66, 0, 1)
        }
    )
    Name(_GPE, Zero) //EC is wired to bit 0 of GPE
    Device(SMB1) {
        Name(_ADR, "ACPI0001")
        Name(_EC, 0x8030) // EC offset(0x80), Query (0x30)
        OperationRegion(PH01, SMBus, 0x65, 0x1)
        Device(DEVB) {
            Name(_ADR, 0x65) // Slave Address 0x65
            Field(PH01, BlockAcc, NoLock, Preserve) {
                AccessAs(BlockAcc, 0),
                FLD1, 128,
                AccessAs(BlockAcc, 0x10),
                FLD2, 32
            }
        } // end of DEVB
    } // end of SMB1
} // end of EC0
```

This example creates a read/write block SMBus device residing at slave address 0x65. There are two fields that use two command registers (0 and 0x10), called “FLD1”, and “FLD2”. Examples of generating the read/write block protocols from ASL is illustrated below:

```
Method(Test) {
    Name(BUF1, Buffer() {8, 1, 2, 3, 4, 5, 6, 7, 8} // 8 is the number of bytes
    Name(BUF2, Buffer() {4, 9, 10, 11, 12} // 4 is the number of bytes
    Store(BUF1, FLD1) // Sets FLD1 SMBus device block register
    Store(BUF2, FLD2) // Sets FLD2 SMBus device block register
    Store(FLD1, Local0) // local0 contains buf: 8,1,2,3,4,5,6,7,8
    Store(FLD2, Local0) // local0 contains buf: 4,9,10,11,12
}
```

15.2.3.3.1.9.3.5 SMBus Memory Devices (*AnyAcc*)

The *AnyAcc* access type allows any of the Read/Write byte, word or Block protocol transfers to be made to the selected SMBus slave address and command value. The combined slave and command value generates a

single byte granular address space. The command address (A0-A7 of the 15-bit address) is defined through the use of the `AccessAs(AccessType, AccessAttribute)` macro. In this case the *AccessAttribute* represents the byte aligned command value, and *AccessType* would be set to *AnyAcc*.

To define a *AnyAcc* device an operation region is generated using the SMBus address type. Next a field is generated in the operation region using the “*AnyAcc*” access type. In the field list an `AccessAs(AnyAcc, command_value)` macro is used to define what command address is associated with this field. The absence of the macro assume a starting command value of 0. The SMBus driver assumes that after the `AccessAs(AnyAcc, command_value)` macro is declared then command registers are byte-granular and linear. If a field is defined that crosses over a byte boundary, then the SMBus driver assumes this field resides in multiple command registers with a command address value of *command_value*+1 (for each new register). The SMBus driver will use the most appropriate protocol for accessing the registers associated with the fields. For example, if a field spans more than three bytes a read/write block protocol access can be made, while if only spanning a byte then the read/write byte protocol can be used.

For example, a 5-byte buffer with the contents of “ACPI” would be generated as:

```
Buffer() {"ACPI"}
```

On reads, ASL will return a buffer with the first entry set to the number of data bytes returned. For example,

```
Device(\_SB.EC0) {
    Name(_HID, EISAID("PNP0C09"))
    Name(_CRS,
        ResourceTemplate() {
            IO(Decode16, 0x62, 0x62, 0, 1),
            IO(Decode16, 0x66, 0x66, 0, 1)
        }
    )
    Name(_GPE, Zero) //EC is wired to bit 0 of GPE
    Device (SMB1) {
        Name(_ADR, "ACPI0001")
        Name(_EC, 0x8030) // EC offset(0x80), Query (0x30)
        OperationRegion(PH01, SMBus, 0x66, 0x1)
        Device(DEVB) {
            Name(_ADR, 0x66) // Slave Address 0x66
            Field(PH01, AnyAcc, NoLock, Preserve) {
                FLD1, 512,
                FLD2, 256,
                FLD3, 32,
                FLD4, 16,
                FLD5, 8
            }
        } // end of DEVB
    } // end of SMB1
} // end of EC0
```

This definition creates a linear SMBus device residing at slave address 0x66. There are six fields that use 102 command registers (0-101), called “FLD1”, “FLD2”, “FLD3”, “FLD4” and “FLD5”. FLD1 references command registers 0-63 (first 64 bytes) and will be accessed by the block protocol (data is over 3 bytes). FLD2 represents command registers 64-95 (next 32 bytes) and will be accessed by the block command protocol (data is over 3 bytes). FLD3 represents command registers 96-99 (next four bytes) and will be accessed by the block command protocol (data is over 3 bytes). FLD4 represents command registers 100-101 (next two bytes) and will be accessed by the word command protocol. FLD5 represents command register 102 (next byte) and will be accessed by the byte command protocol. Examples of generating the accesses from ASL is illustrated below:

```

Method(Test){
    Name(BUF1, Buffer()){“Hannibal”}
    Name(BUF2, Buffer()){“Scipio Africanus”}
    Name(BUF3, Buffer()){“Zama”}
    Store(BUF1, FLD1)        // writes “Hannibal” to linear addresses for FLD1
    Store(BUF2, FLD2)        // writes “Scipio Africanus” to linear addresses for FLD2
    Store(BUF3, FLD3)        // writes “Zama” to linear addresses for FLD3
    Store(0xFF12, FLD4)       // sets FLD4 to 0xFF12
    Store(0xEF, FLD5)         // sets FLD5 to 0xEF
    Store(FLD1, Local0)       // local0 contains 64 byte buffer with: “Hannibal”,0,...
    Store(FLD2, Local0)       // local0 contains 32 byte buffer with: “Scipio Africanus”,0,...
    Store(FLD3, Local0)       // local0 contains 4 bytes: “Zama”
    Store(FLD4, Local0)       // local0 contains 2 bytes: 0xFF12
    Store(FLD5, Local0)       // local0 contains 1 byte: 0xEF
}

```

15.2.3.3.1.9.3.6 Mixed Example (AnyAcc)

Some devices can be accessed through multiple protocols. This section gives an example of such a device.

```

Device(\SB_EC0) {
    Name(_HID, EISAID("PNP0C09"))
    Name(_CRS,
        ResourceTemplate(){
            IO(Decode16, 0x62, 0x62, 0, 1),
            IO(Decode16, 0x66, 0x66, 0, 1)
        }
    )
    Name(_GPE, Zero)           //EC is wired to bit 0 of GPE
    Device(SMB1) {
        Name(_ADR, "ACPI0001")
        Name(_EC, 0x8030)      // EC offset(0x80), Query (0x30)
        OperationRegion(PH01, SMBus, 0x67, 0x1)
        Device(DEVB) {
            Name(_ADR, 0x67)    // Slave Address 0x67
            Field(PH01, ByteAcc, NoLock, Preserve) {
                AccessAs(AnyAcc, 0),
                FLD1, 512,
                FLD2, 256,
                FLD3, 32,
                AccessAs(WordAcc, 0x70),
                FLD4, 16,
                AccessAs(ByteAcc, 0x80),
                FLD5, 8
            }
        }
    }
}
// end of DEVB
// end of SMB1
// end of EC0

```

This definition creates an SMBus device using various protocols residing at slave address 0x67. There are three fields that use four command registers (0, 1, 2 and 3), called “FLD1”, “FLD2” and “FLD3”. FLD1 references command registers 0-1 (32 bytes per command register) and will be accessed by the byte, word and block linear protocols. FLD2 represents command register 064 and will be accessed by the byte, word and block linear protocols. FLD3 represents command register 96 and will be accessed by the byte, word and block linear protocols. FLD4 represents command register 0x70 and will be accessed by the word command protocol. FLD5 represents command register 0x80 and will be accessed by the byte command protocol.

15.2.3.3.1.10 IndexField - Declare Index/Data Fields

```

IndexFieldTerm      := IndexField(
                        IndexName,           //NameString
                        DataName,           //NameString
                        AccessType,         //AccessTypeKeyword
                        LockRule,           //LockRuleKeyword
                        UpdateRule,         //UpdateRuleKeyword
                    ) {FieldUnitList}

```

Creates a series of named data objects whose data values are fields within a larger object accessed by an index/data-style reference to *IndexName* and *DataName*.

This encoding is used to define named data objects whose data values are fields within an index/data register pair. This provides a simple way to declare register variables that occur behind a typical index and data register pair.

Accessing the contents of an indexed field data object will automatically occur through the *DataName* object by using an *IndexName* object aligned on an *AccessType* boundary, with synchronization occurring on the operation region which contains the index data variable, and on the global lock if specified by *LockRule*.

AccessType, *LockRule*, *UpdateRule*, and *FieldList* are the same format as the *Field* term.

The following is a block of ASL sample code using **IndexField**:

- Creates an index/data register in system I/O space made up of 8-bit registers.
- Creates a FET0 field within the indexed range.

```
Method( _EX1){
    // define 256-byte operational region in SystemIO space
    // and name it GIO0
    OperationRegion (GIO0, 1, 0x125, 0x100)
    // create field named Preserve structured as a sequence
    // of index and data bytes
    Field (GIO0, ByteAcc, NoLock, WriteAsZeros) {
        IDX0, 8,
        DAT0, 8,
        .
        .
        .
    }
    // Create an IndexField within IDX0 & DAT0 which has
    // FETs in the first two bits of indexed offset 0,
    // and another 2 FETs in the high bit on indexed
    // 2f and the low bit of indexed offset 30
    IndexField (IDX0, DAT0, ByteAcc, NoLock, Preserve) {
        FET0, 1,
        FET1, 1,
        Offset(0x2f),      // skip to byte offset 2f
        , 7,               // skip another 7 bits
        FET3, 1,
        FET4, 1
    }
    // Clear FET3 (index 2f, bit 7)
    Store (Zero, FET3)
}
```

15.2.3.3.1.11 Method - Declare Control Method

```
MethodTerm ::= Method(
    MethodName,           //NameString
    NumArgs,              //Nothing | ByteConst
    SerializeRule         //Nothing |
                        //SerializeRuleKeyword
) {TermList}
```

Declares a named package containing a series of object references that collectively represent a control method, which is a procedure that can be invoked to perform computation. **Method** opens a name scope.

System software executes a control method by referencing the objects in the package in order. For more information on control method execution, see section 5.5.3.

The current name space location used during name creation is adjusted to be the current location on the name space tree. Any names created within this scope are “below” the name of this package. The current name space location is assigned to the method package, and all name space references that occur during control method execution for this package are relative to that location.

If a method is declared as **Serialized**, an implicit mutex associated with the method object is acquired at SyncLevel 0. The serialize-rule can be used to prevent re-entering of a method. This is especially useful if the method creates name space objects. Without the serialize-rule, the re-entering of a method will fail when it attempts to create the same name space object.

Also note that all name space objects created by a method have temporary lifetime. When method execution exits, the created objects will be destroyed.

The following block of ASL sample code shows a use of **Method** for defining a control method that turns on a power resource.

```
Method(_ON) {
    Store (One, GPIO.IDEP)      // assert power
    Sleep (10)                  // wait 10ms
    Store (One, GPIO.IDER)      // de-assert reset#
    Stall (10)                  // wait 10us
    Store (Zero, GPIO.IDEI)     // de-assert isolation
}
```

15.2.3.3.1.12 Mutex - Declare Synchronization / Mutex Object

```
MutexTerm          := Mutex(
                        MutexName,          //NameString
                        SyncLevel           //ByteConst
                    )
```

Creates a data mutex synchronization object named *MutexName*, with level from 0 to 15 specified by *SyncLevel*.

A synchronization object provides a control method with a mechanism for waiting for certain events. To prevent deadlocks, wherever more than one synchronization object must be owned, the synchronization objects must always be released in the order opposite the order in which they were acquired. The *SyncLevel* parameter declares the logical nesting level of the synchronization object. All **Acquire** terms must refer to a synchronization object with an equal or greater *SyncLevel* to current level, and all **Release** terms must refer to a synchronization object with equal or lower *SyncLevel* to the current level.

Mutex synchronization provides the means for mutually exclusive ownership. Ownership is acquired using an **Acquire** term and is released using a **Release** term. Ownership of a Mutex must be relinquished before completion of any invocation. For example, the top level control method cannot exit while still holding ownership of a Mutex. Acquiring ownership of a Mutex can be nested. The *SyncLevel* check is not performed on a Mutex when the ownership count is nesting.

The *SyncLevel* of a thread before acquiring any mutexes is zero. The *SyncLevel* of the global lock (_GL) is zero. A method marked serialized has an inherent mutex of *SyncLevel* 0.

15.2.3.3.1.13 OperationRegion - Declare Operation Region

```
OpRegionTerm       := OperationRegion(
                        RegionName,          //NameString
                        RegionSpace,        //RegionSpaceKeyword
                        Offset,             //TermArg=>DWordConst
                        Length              //TermArg=>DWordConst
                    )
```

Declares an operation region. *Offset* is the offset within the selected *RegionSpace* at which the region starts (byte-granular), and *Length* is the length of the region in bytes.

An Operation Region is a type of data object where read or write operations to the data object are performed in some hardware space. For example, the Definition Block can define an Operation Region within a bus, or system IO space. Any reads or writes to the named object will results in accesses to the IO space.

Operation regions are regions in some space that contain hardware registers for *exclusive* use by ACPI control methods. In general, no hardware register (at least byte granular) within the operation region accessed by an ACPI control method can be shared with any accesses from any other source, with the exception of using the Global Lock to share a region with the firmware. The entire Operation Region can be allocated for exclusive use to the ACPI subsystem in the host OS.

Operation Regions have “virtual content” and are only accessible via **Field** objects Operation Region objects may be defined down to actual bit controls using **Field** data object definitions. The actual bit content of a **Field** are bits from within a larger **Buffer** that are normalized for that field (i.e., shifted down and masked to the

proper length), and as such the data type of a **Field** is **Buffer**. Therefore fields which are 32 bits or less in size may be read and stored as Integers.

An Operation Region object implicitly supports Mutex synchronization. Updates to the object, or a Field data object for the region, will automatically synchronize on the Operation Region object; however, a control method may also explicitly synchronize to a region to prevent other accesses to the region (from other control methods). Note that, according to the control method execution model, control method execution is non-preemptive. Because of this, explicit synchronization to an Operation Region needs to be done only in cases where a control method blocks or yields execution and where the type of register usage requires such synchronization.

Originally there were five Operation Region types specified in ACPI:

- 0 = SystemMemory
- 1 = SystemIO
- 2 = PCI_Config
- 3 = EmbeddedControl
- 4 = SMBus

These are now extended to include vendor-defined Operation Regions, with 0x80 to 0xFF user defined.

The following example ASL code shows the use of **OperationRegion** combined with Field to describe IDE 0 and 1 controlled through general IO space, using one FET.

```
OperationRegion (GIO, SystemIO, 0x125, 0x1)
Field (GIO, ByteAcc, NoLock, Preserve) {
    IDEI, 1,      // IDEISO_EN   - isolation buffer
    IDEP, 1,      // IDE_PWR_EN   - power
    IDER, 1       // IDERST#_EN   - reset#
}
```

15.2.3.3.1.14 PowerResource - Declare Power Resource

```
PowerResTerm          := PowerResource (
    ResourceName,      //NameString
    SystemLevel,       //ByteConst
    ResourceOrder,     //WordConst
) {ObjectList}
```

Declares a power resource. **PowerResource** opens a name scope.

For a definition of the **PowerResource** term, see section 7.1.

15.2.3.3.1.15 Processor - Declare Processor

```
ProcessorTerm         := Processor (
    ProcessorName,     //NameString
    ProcessorID,       //ByteConst
    PBlockAddress,     //DWordConst
    PblockLength,      //ByteConst
) {ObjectList}
```

Declares a named processor object. **Processor** opens a name scope. Each processor is required to have a unique *ProcessorID* value from any other *ProcessorID* value.

The ACPI BIOS declares one processor object per processor in the system under the _PR name space. *PBlockAddress* provides the system IO address for the processors register block. Each processor can supply a different such address. *PBlockLength* is the length of the processor register block, in bytes which is either 0 (for no P_BLK) or 6. With one exception, all processors are required to have the same *PBlockLength*. The exception is that the boot processor can have a non-zero *PBlockLength* when all other processors have a zero *PBlockLength*.

The following block of ASL sample code shows a use of the **Processor** term.

```
Processor(
    \_PR.CPU0,      // name space name
    1,
    0x120,          // PBlk system IO address
    6              // PBlkLen
)
{ }
```

15.2.3.3.1.16 ThermalZone - Declare Thermal Zone

```
ThermalZoneTerm      := ThermalZone(
                        ThermalZoneName      //NameString
                        ) {ObjectList}
```

Declares a named Thermal Zone object. **ThermalZone** opens a name scope.

Each use of a **ThermalZone** term declares one thermal zone in the system. Each thermal zone in a system is required to have a unique *ThermalZoneName*.

For sample ASL code that uses a ThermalZone statement, see section **Error! Reference source not found..**

15.2.3.3.2 Name Space Modifiers

The name space modifiers are as follows:

Table 15-9 Name Space Modifiers

ASL Statement	Description
Alias	Defines a name alias
Name	Defines a global name and attaches a buffer, literal data item, or package to it.
Scope	Declares the placement of one or more object names in the ACPI name space when the definition block that contains the Scope statement is loaded.

15.2.3.3.2.1 Alias - Declare Name Alias

```
AliasTerm      := Alias(
                  SourceObject,      //NameString
                  AliasObject        //NameString
                )
```

Creates a new name, *AliasObject*, which refers to and acts exactly the same as *SourceObject*.

AliasObject is created as an alias of *SourceObject* in the name space. The *SourceObject* name must already exist in the name space. If the alias is to a name within the same definition block the *SourceObject* name must be logically ahead of this definition in the block. The following example shows use of an **Alias** term:

```
Alias(\SUS.SET.EVEN, SSE)
```

15.2.3.3.2.2 Name - Declare Named Object

```
NameTerm      := Name(
                  ObjectName,      //NameString
                  Object            //DataObject
                )
```

Attaches *Object* to *ObjectName* in the Global ACPI name space.

This encoding is to create *ObjectName* in the name space, which references the *Object*.

The following example creates the name PTTX in the root of the name space that references a package.

```

Name(\PTTX, // Port to Port Translate Table
    Package() { Package() { 0x43, 0x59 }, Package() { 0x90, 0xff }}
)

```

The following example creates the name CNT in the root of the name space that references an integer data object with the value 5.

```

Name(\CNT, 5)

```

15.2.3.3.2.3 Scope - Declare Name Scope

```

ScopeTerm      := Scope(
                    Location //NameString
                ) {ObjectList}

```

Gives a base scope to a collection of objects. All object names defined within the scope act relative to *Location*. Note that *Location* does not have to be below the surrounding scope. Note also that the **Scope** term does not create objects, but only locates objects in the name space; the located objects are created by other ASL terms.

The **Scope** term alters the current name space location to *Location*. This causes the defined objects within *TermList* to occur relative to the new location in the name space.

The following example ASL code

```

Scope(\PCI0) {
    Name(X, 3)
    Scope(\) {
        Method(_RQ) { Return(0) }
    }
    Name(^Y, 4)
}

```

places the defined objects in ACPI name space as shown in the following:

```

\PCI0.X
\ _RQ
\ ^Y

```

15.2.3.4 Opcode Terms

There are two types of ASL opcode terms: Type 1 opcodes and Type 2 opcodes.

- A Type1 opcode term can only be used standing alone on a line of ASL code; because these types of terms do not return a value, they cannot be used as a term in an expression.
- A Type2 opcode term can be used in an expression because these types of terms return a value. When used in an expression the argument that names the object in which to store the result can be optional.

Note that in the opcode definitions below, when the definition says “result is stored in” this literally means that the **Store** operator is assumed and the “execution result” is the **Source** operand to the **Store** opcode.

15.2.3.4.1 Type 1 Opcodes

```

Type1Opcode    := BreakTerm | BreakPointTerm | FatalTerm | IfElseTerm |
                    LoadTerm | NoOpTerm | NotifyTerm | ReleaseTerm | ResetTerm |
                    ReturnTerm | SignalTerm | SleepTerm | StallTerm | UnloadTerm
                    | WhileTerm

```

The Type 1 opcodes are listed in the following table.

Table 15-10 Type 1 Opcodes

ASL Statement	Description
Break	Stop executing the current code package at this point
BreakPoint	Used for debugging. Stops execution in the debugger
Else	Else

ASL Statement	Description
Fatal	Fatal check
If	If
Load	Load differentiating definition block
Noop	No operation
Notify	Notify the OS that a specified notification value for a NotifyObject has occurred
Release	Release a synchronization object
Reset	Reset a synchronization object
Return	Return from a control method, optionally setting a return value
Signal	Signal a synchronization object
Sleep	Sleep n milliseconds (yields the processor)
Stall	Delay n microseconds (does not yield the processor)
Unload	Unload differentiating definition block
While	While

15.2.3.4.1.1 Break - Break

BreakTerm := **Break**

The break operation causes the current package execution to complete.

15.2.3.4.1.2 BreakPoint - BreakPoint

BreakPointTerm := **BreakPoint**

Used for debugging, the **Breakpoint** opcode stops the execution and enters the AML debugger. In the retail version of the interpreter, **BreakPoint** is equivalent to **Noop**.

15.2.3.4.1.3 Else - Else Operator

ElseTerm := Nothing | <**Else** {TermList}>

In an **If** term, if *Predicate* evaluates to 0, it is false, and the term list in the **Else** term is executed. If *Predicate* evaluates to Not 0 on the **If** term, then it is considered true, and the term list in the **Else** term is not executed.

The following example checks Local0 to be zero or non-zero. On non-zero, CNT is incremented; otherwise, CNT is decremented.

```
If (Local0) {
    Increment (CNT)
} Else {
    Decrement (CNT)
}
```

15.2.3.4.1.4 Fatal - Fatal Check

```
FatalTerm := Fatal(
    Type, //ByteConst
    Code, //DWordConst
    Arg //TermArg=>Integer
)
```

This operation is used to inform the OS that there has been an OEM-defined fatal error. In response, the OS must log the fatal event and perform a controlled OS shutdown in a timely fashion.

15.2.3.4.1.5 If – If Operator

```
IfTerm := If(
    Predicate //TermArg=>Integer
) {TermList}
```

Predicate is evaluated as an integer. If the integer is non-zero, the term list of the **If** term is executed.

The following examples all check for bit 3 in **Local0** being set, and clear it if set.

```

// example 1
if (And(Local0, 4)) {
    XOR (Local0, 4, Local0)
}
// example 2
Store(4, Local2)
if (And(Local0, Local2)) {
    XOR (Local0, Local2, Local0)
}

```

15.2.3.4.1.6 Load - Load Differentiated Definition Block

```

LoadTerm                               := Load(
                                     Object,           //NameString
                                     DDBHandle          //SuperName
                                     )

```

Performs a run time load of a Definition Block. The *Object* parameter can either refer to an operation region field or an operation region directly. If the object is an operation region, the operation region must be in SystemMemory space. The Definition Block should contain a DESCRIPTION_HEADER of type SSDT or PSDT. The Definition Block must be totally contained within the supplied operational region or operation region field. This table is read into memory, the checksum is verified, and then it is loaded into the ACPI name space. The *DDBHandle* parameter is the handle to the Differentiating Definition Block that can be used to unload the Definition Block at a future time.

The OS can also check the OEM Table ID and Revision ID against a database for a newer revision Definition Block of the same OEM Table ID and load it instead.

The default name space location to load the Definition Block is relative to the current name space. The new Definition Block can override this by specifying absolute names or by adjusting the name space location using the **Scope** operator.

Loading a Definition Block is a synchronous operation. Upon completion of the operation, the Definition Block has been loaded. The control methods defined in the Definition Block are not executed during load time.

15.2.3.4.1.7 Noop Code - No Operation

```

NoOpTerm                               := Noop

```

This operation has no effect.

15.2.3.4.1.8 Notify - Notify

```

NotifyTerm                             := Notify(
                                     Object,           //SuperName
                                     NotificationValue //TermArg=>ByteConst
                                     )

```

Notifies the OS that the *NotificationValue* for the *Object* has occurred. *Object* must be a reference to a device or thermal zone object.

Notification values are determined by the *Object* type. For example, the notify values for a thermal zone object are different from the notify values used for a device object. Undefined notification values are treated as reserved and are ignored by the OS.

For lists of defined Notification values, see section **Error! Reference source not found..**

15.2.3.4.1.9 Release - Release a Mutex Synchronization Object

```

ReleaseTerm                             := Release(
                                     SyncObject        //SuperName
                                     )

```

SyncObject must be a mutex synchronization object. If the mutex object is owned by the current invocation, ownership for the Mutex is released once. It is fatal to release ownership on a Mutex unless it is currently owned. A Mutex must be totally released before an invocation completes.

15.2.3.4.1.10 Reset - Reset an Event Synchronization Object

```

ResetTerm                               := Reset(
                                     SyncObject        //SuperName
                                     )

```

SynchObject must be an Event synchronization object. This encoding is used to reset an event synchronization object to a non-signaled state. See also the Wait and Signal function operator definitions.

15.2.3.4.1.11 Return - Return

```
ReturnTerm      := Return(
                    Arg                                //TermArg=>DataObject
                )
```

Returns control to the invoking control method, optionally returning a copy of the object named in *Arg*.

15.2.3.4.1.12 Signal - Signal a Synchronization Event

```
SignalTerm      := Signal(
                    SynchObject                        //SuperName
                )
```

SynchObject must be an Event synchronization object. The Event object is signaled once, allowing one invocation to acquire the event.

15.2.3.4.1.13 Sleep - Sleep

```
SleepTerm       := Sleep(
                    MilliSecs                          //TermArg=>Integer
                )
```

The **Sleep** term is used to implement long-term timing requirements. Execution is delayed for at least the required number of milliseconds. The implementation of **Sleep** is to round the request up to the closest sleep time supported by the OS and relinquish the processor.

15.2.3.4.1.14 Stall - Stall for a Short Time

```
StallTerm       := Stall(
                    MicroSecs                          //TermArg=>Integer
                )
```

The **Stall** term is used to implement short-term timing requirements. Execution is delayed for at least the required number of microseconds. The implementation of **Stall** is OS-specific, but must not relinquish control of the processor. Because of this, delays longer than 100 microseconds must use **Sleep** instead of **Stall**.

15.2.3.4.1.15 Unload - Unload Differentiated Definition Block

```
UnloadTerm      := Unload(
                    DDBHandle                          //SuperName
                )
```

Performs a run time unload of a Definition Block that was loaded using a **Load** term. Loading or unloading a Definition Block is a synchronous operation, and no control method execution occurs during the function. On completion of the **Unload** operation, the Definition Block has been unloaded (all the name space objects created as a result of the corresponding Load operation will be removed from the name space).

15.2.3.4.1.16 While - While

```
WhileTerm       := While(
                    Predicate                          //TermArg=>Integer
                ) {TermList}
```

Predicate is evaluated as an integer. If the integer is non-zero, the list of terms in *TermList* is executed. The operation repeats until the *Predicate* evaluates to zero.

15.2.3.4.2 Type 2 Opcodes

```
Type2Opcode     := AcquireTerm | AddTerm | AndTerm | ConcatTerm | CondRefOfTerm
                  | DecTerm | DerefOfTerm | DivideTerm | FindSetLeftBitTerm | |
                  | FindSetRightBitTerm | FromBCDTerm | IncTerm | IndexTerm |
                  | LAndTerm | LEqualTerm | LGreaterTerm | LGreaterEqualTerm |
                  | LLessTerm | LLessEqualTerm | LNotTerm | LNotEqualTerm |
                  | LOrTerm | MatchTerm | MultiplyTerm | NAndTerm | NOrTerm |
                  | NotTerm | ObjectTypeTerm | OrTerm | RefOfTerm |
                  | ShiftLeftTerm | ShiftRightTerm | SizeOfTerm | StoreTerm |
                  | SubtractTerm | ToBCDTerm | WaitTerm | XorTerm | UserTerm
```

The ASL terms for Type 2 Opcodes are listed in the following table.

Table 15-11 Type 2 Opcodes

ASL Statement	Description
Acquire	Acquire a synchronization object
Add	Add two values
And	Bitwise And
Concatenate	Concatenate two strings
CondRefOf	Conditional reference to an object
Decrement	Decrement a value.
DerefOf	Dereference of an object reference
Divide	Divide
FindSetLeftBit	Index of first set Lsb
FindSetRightBit	Index of first set Msb
FromBCD	Convert from BCD to numeric
Increment	Increment a value
Index	Reference the nth element of a package
LAnd	Logical And
LEqual	Logical Equal
LGreater	Logical Greater
LGreaterEqual	Logical Not less
LLess	Logical Less
LLessEqual	Logical Not greater
LNot	Logical Not
LNotEqual	Logical Not equal
LOr	Logical Or
Match	Search for match in package array
Multiply	Multiply
NAnd	Bitwise Nand
NOr	Bitwise Nor
Not	Bitwise Not
ObjectType	Type of object
Or	Bitwise Or
RefOf	Reference to an object
ShiftLeft	Shift value left
ShiftRight	Shift value right
SizeOf	Get the size of a buffer, string, or package
Store	Store value
Subtract	Subtract values
ToBCD	Convert numeric to BCD
Wait	Wait
Xor	Bitwise Xor

15.2.3.4.2.1 Acquire - Acquire a Mutex Synchronization Object

```

AcquireTerm      := Acquire (
                    SynchronObject,           //SuperName
                    TimeoutValue              //WordConst
                ) => Boolean                  //Ones means timed-out

```

SynchronObject must be a mutex synchronization object. It refers to the mutex to be acquired.

Ownership of the Mutex is obtained. If the Mutex is already owned by a different invocation, the processor is relinquished until the owner of the Mutex releases it or until at least *TimeoutValue* milliseconds have elapsed. A Mutex can be acquired more than once by the same invocation.

This operation returns a non-zero value if a timeout occurred and the mutex ownership was not acquired. A *TimeoutValue* of 0xFFFF indicates that there is no time out and the operation will wait indefinitely.

15.2.3.4.2.2 Add - Add

```
AddTerm := Add(
    Addend1, //TermArg=>Integer
    Addend2, //TermArg=>Integer
    Result   //Nothing | SuperName
) => Integer
```

Addend1 and *Addend2* are evaluated as integer data types and are added, and the result is optionally stored into *Result*. Overflow conditions are ignored.

15.2.3.4.2.3 And - Bitwise And

```
AndTerm := And(
    Source1, //TermArg=>Integer
    Source2, //TermArg=>Integer
    Result   //Nothing | SuperName
) => Integer
```

Source1 and *Source2* are evaluated as integer data types, a bit-wise **AND** is performed, and the result is optionally stored into *Result*.

15.2.3.4.2.4 Concatenate - Concatenate

```
ConcatTerm := Concatenate(
    Source1, //TermArg=>ComputationalData
    Source2, //TermArg=>ComputationalData
    Result   //Nothing | SuperName
) => ComputationalData
```

Source1 and *Source2* are evaluated. *Source1* and *Source2* must be of the same data type (that is, both integers, both strings, or both buffers). *Source2* is concatenated to *Source1* and the result data is optionally stored into *Result*.

Table 15-12 Concatenate Data Types

Source1 Data Type	Source2 Data Type	Result Data Type
Integer	Integer	Buffer
String	String	String
Buffer	Buffer	Buffer

15.2.3.4.2.5 CondRefOf - Conditional Reference Of

```
CondRefOfTerm := CondRefOf(
    Source, //SuperName
    Destination //SuperName
) => Boolean
```

Attempts to set *Destination* to refer to *Source*. The *Source* of this operation can be any object type (e.g., data package, device object, etc.). On success, the *Destination* object is set to refer to *Source* and the execution result of this operation is the constant **Ones** object. On failure the execution result of this operation is the constant **Zero** object and the *Destination* object is unchanged. This can be used to reference items in the name space which may appear dynamically (e.g., from a dynamically loaded differentiation definition block).

CondRefOf is equivalent to **RefOf** except that if the *Source* object does not exist, it is fatal for **RefOf** but not for **CondRefOf**.

15.2.3.4.2.6 Decrement - Decrement

```
DecTerm := Decrement(
    Addend //SuperName
) => Integer
```

This operation decrement the *Addend* by one and the result is stored back to *Addend*.

15.2.3.4.2.7 DerefOf – Dereference Of Operator

```
DerefOfTerm := DerefOf(  
    Source //TermArg=>ObjectReference  
) => ObjectReference
```

Returns the object referred by the *Source* object reference. The object returned can be any object type (for example, a package, a device object, and so on).

15.2.3.4.2.8 Divide - Divide

```
DivideTerm := Divide(  
    Dividend, //TermArg=>Integer  
    Divisor, //TermArg=>Integer  
    Remainder, //Nothing | SuperName  
    Result //Nothing | SuperName  
) => Integer //returns Result
```

Dividend and *Divisor* are evaluated as integer data. *Dividend* is divided by *Divisor*, then the resulting remainder is optionally stored into *Remainder* and the resulting quotient is optionally stored into *Result*. Divide-by-zero exceptions are fatal.

15.2.3.4.2.9 FindSetLeftBit – Find Set Left Bit

```
FindSetLeftBitTerm := FindSetLeftBit(  
    Source, //TermArg=>Integer  
    Result //Nothing | SuperName  
) => Integer
```

Source is evaluated as integer data type, and the one-based bit location of the first MSb (most significant set bit) is optionally stored into *Result*. The result of 0 means no bit was set, 1 means the left-most bit set is the first bit, 2 means the left-most bit set is the second bit, and so on.

15.2.3.4.2.10 FindSetRightBit - Find Set Right Bit

```
FindSetRightBitTerm := FindSetRightBit(  
    Source, //TermArg=>Integer  
    Result //Nothing | SuperName  
) => Integer
```

Source is evaluated as integer data type, and the one-based bit location of the most LSb (least significant set bit) is optionally stored in *Result*. The result of 0 means no bit was set, 32 means the first bit set is the 32nd bit, 31 means the first bit set is the 31st bit, and so on.

15.2.3.4.2.11 FromBCD - Convert from BCD

```
FromBCDTerm := FromBCD(  
    BCDValue, //TermArg=>Integer  
    Result //Nothing | SuperName  
) => Integer
```

The **FromBCD** operation is used to convert *BCDValue* to a numeric format and store the numeric value in *Result*.

15.2.3.4.2.12 Increment - Increment

```
IncTerm := Increment(  
    Addend //SuperName  
) => Integer
```

Equivalent to **Add**(*Addend*, 1, *Addend*)

15.2.3.4.2.13 Index - Index

```
IndexTerm := Index(  
    Source, //TermArg=>  
    //<BufferTerm | PackageTerm>  
    Index, //TermArg=>Integer  
    Destination //Nothing | SuperName  
) => ObjectReference
```

Source is evaluated to either buffer or package data type. *Index* is evaluated to an integer. The object at *Index* within *Source* is optionally stored as a reference into *Destination*. The following example ASL code shows a way to use the **Index** term to store into a local variable the sixth element of the first package of a set of nested packages:

```
Name(IO0D, Package() {
    Package() {
        0x01, 0x03F8, 0x03F8, 0x01, 0x08, 0x01,
        0x25, 0xFF, 0xFE, 0x00, 0x00
    },
    Package() {
        0x01, 0x02F8, 0x02F8, 0x01, 0x08, 0x01,
        0x25, 0xFF, 0xBE, 0x00, 0x00
    },
    Package() {
        0x01, 0x03E8, 0x03E8, 0x01, 0x08, 0x01,
        0x25, 0xFF, 0xFA, 0x00, 0x00
    },
    Package() {
        0x01, 0x02E8, 0x02E8, 0x01, 0x08, 0x01,
        0x25, 0xFF, 0xBA, 0x00, 0x00
    },
    Package() {
        0x01, 0x0100, 0x03F8, 0x08, 0x08, 0x02,
        0x25, 0x20, 0x7F, 0x00, 0x00,
    }
})
//Get the 6th element of the first package
Store(DeRefOf(Index(DeRefOf(Index(IO0D, 0)), 5)), Local0)
```

The following example ASL code shows a way to store into the 3rd byte of a buffer:

```
Name(BUFF, Buffer() {
    0x01, 0x02, 0x03, 0x04, 0x05
})
//Store 0x55 into the third byte of the buffer
Store(0x55, Index(BUFF, 2))
```

15.2.3.4.2.14 LAnd - Logical And

```
LAndTerm := LAnd(
    Source1, //TermArg=>Integer
    Source2 //TermArg=>Integer
) => Boolean
```

Source1 and *source2* are evaluated as integers. If both values are non-zero, the constant object **Ones** is returned, otherwise the constant object **Zero** is returned.

15.2.3.4.2.15 LEqual - Logical Equal

```
LEqualTerm := LEqual(
    Source1, //TermArg=>Integer
    Source2 //TermArg=>Integer
) => Boolean
```

Source1 and *Source2* are evaluated as integers. If the values are equal, the constant object **Ones** is returned; otherwise, the constant object **Zero** is returned.

15.2.3.4.2.16 LGreater - Logical Greater

```
LGreaterTerm := LGreater(
    Source1, //TermArg=>Integer
    Source2 //TermArg=>Integer
) => Boolean
```

Source1 and *Source2* are evaluated as integers. If *Source1* is greater than *Source2*, the constant object **Ones** is returned; otherwise, the constant object **Zero** is returned.

15.2.3.4.2.17 LGreaterEqual - Logical Greater Than Or Equal

```
LGreaterEqualTerm := LGreaterEqual(
    Source1, //TermArg=>Integer
    Source2 //TermArg=>Integer
) => Boolean
```

Source1 and *Source2* are evaluated as integers. If *Source1* is greater than or equal to *Source2*, the constant object **Ones** is returned; otherwise, the constant object **Zero** is returned.

15.2.3.4.2.18 LLess - Logical Less

```
LLessTerm                := LLess(
                            Source1,           //TermArg=>Integer
                            Source2           //TermArg=>Integer
                        ) => Boolean
```

Source1 and *Source2* are evaluated as integers. If *Source1* is less than *Source2*, the constant object **Ones** is returned; otherwise, the constant object **Zero** is returned.

15.2.3.4.2.19 LLessEqual - Logical Less Than Or Equal

```
LLessEqualTerm           := LLessEqual(
                            Source1,           //TermArg=>Integer
                            Source2           //TermArg=>Integer
                        ) => Boolean
```

Source1 and *Source2* are evaluated as integers. If *Source1* is less than or equal to *Source2*, then the constant object **Ones** is returned; otherwise, the constant object **Zero** is returned.

15.2.3.4.2.20 LNot - Logical Not

```
LNotTerm                 := LNot(
                            Source,           //TermArg=>Integer
                        ) => Boolean
```

Source1 is evaluated as an integer. If the value is non-zero, the constant object **Zero** is returned; otherwise, the constant object **Ones** is returned.

15.2.3.4.2.21 LNotEqual - Logical Not Equal

```
LNotEqualTerm            := LNotEqual(
                            Source1,           //TermArg=>Integer
                            Source2           //TermArg=>Integer
                        ) => Boolean
```

Source1 and *Source2* are evaluated as integers. If *Source1* is not equal to *Source2*, then the constant object **Ones** is returned; otherwise, the constant object **Zero** is returned.

15.2.3.4.2.22 LOr - Logical Or

```
LOrTerm                  := LOr(
                            Source1,           //TermArg=>Integer
                            Source2           //TermArg=>Integer
                        ) => Boolean
```

Source1 and *Source2* are evaluated as integers. If either values is non-zero, the constant object **Ones** is returned; otherwise, the constant object **Zero** is returned.

15.2.3.4.2.23 Match - Find Object Match

```
MatchTerm                := Match(
                            SearchPackage,     //TermArg=>Package
                            Op1,               //MatchOpKeyword
                            MatchObject1,     //TermArg=>Integer
                            Op2,               //MatchOpKeyword
                            MatchObject2,     //TermArg=>Integer
                            StartIndex        //TermArg=>Integer
                        ) => Ones | Integer
```

SearchPackage is evaluated to a package object and is treated as a one-dimension array. A comparison is performed for each element of the package, starting with the index value indicated by *StartIndex* (0 is the first element). If the element of *SearchPackage* being compared against is called *P[i]*, then the comparison is:

if (*P[i]* *Op1* *MatchObject1*) and (*P[i]* *Op2* *MatchObject2*) then Match => i is returned.

If the comparison succeeds, the index of the element that succeeded is returned; otherwise, the constant object **Ones** is returned.

Op1 and *Op2* have the following values and meanings listed in the following table.

Table 15-13 Match Term Operator Meanings

Operator	Encoding	Macro
TRUE - a don't care, always returns TRUE	0	MTR
EQ - returns TRUE if P[i] == MatchObject	1	MEQ
LE - returns TRUE if P[i] <= MatchObject	2	MLE
LT - returns TRUE if P[i] < MatchObject	3	MLT
GE - returns TRUE if P[i] >= MatchObject	4	MGE
GT - returns TRUE if P[i] > MatchObject	5	MGT

Following are some example uses of Match:

```
Name(P1,
Package() {1981, 1983, 1985, 1987, 1989, 1990, 1991, 1993, 1995, 1997, 1999, 2001}
)

// match 1993 == P1[i]
Match(P1, MEQ, 1993, MTR, 0, 0) // -> 7, since P1[7] == 1993

// match 1984 == P1[i]
Match(P1, MEQ, 1984, MTR, 0, 0) // -> ONES (not found)

// match P1[i] > 1984 and P1[i] <= 2000
Match(P1, MGT, 1984, MLE, 2000, 0) // -> 2, since P1[2]>1984 and P1[2]<=2000

// match P1[i] > 1984 and P1[i] <= 2000, starting with 3rd element
Match(P1, MGT, 1984, MLE, 2000, 3) // -> 3, first match at or past Start
```

15.2.3.4.24 Multiply - Multiply

```
MultiplyTerm := Multiply(
    Multiplicand, //TermArg=>Integer
    Multiplier, //TermArg=>Integer
    Result //Nothing | SuperName
) => Integer
```

Multiplicand and *Multiplier* are evaluated as integer data types. *Multiplicand* is multiplied by *Multiplier*, and the result is optionally stored into *Result*. Overflow conditions are ignored.

15.2.3.4.25 NAnd - Bit-wise NAnd

```
NAndTerm := NAnd(
    Source1, //TermArg=>Integer
    Source2, //TermArg=>Integer
    Result //Nothing | SuperName
) => Integer
```

Source1 and *Source2* are evaluated as integer data types, a bit-wise **NAND** is performed, and the result is optionally stored in *Result*.

15.2.3.4.26 NOR - Bitwise NOR

```
NORTerm := NOR(
    Source1, //TermArg=>Integer
    Source2, //TermArg=>Integer
    Result //Nothing | SuperName
) => Integer
```

Source1 and *Source2* are evaluated as integer data types, a bit-wise **NOR** is performed, and the result is optionally stored in *Result*.

15.2.3.4.27 Not - Not

```
NotTerm := Not(
    Source, //TermArg=>Integer
    Result //Nothing | SuperName
) => Integer
```

Source1 is evaluated as an integer data type, a bit-wise **NOT** is performed, and the result is optionally stored in *Result*.

15.2.3.4.2.28 ObjectType - Object Type

```
ObjectTypeTerm      := ObjectType(  
                        Object                               //SuperName  
                    ) => Integer
```

The execution result of this operation is an integer that has the numeric value of the object type for *Object*. The object type codes are listed in the following table. Note that if this operation is performed on an object reference such as one produced by the **Alias**, **Index** or **RefOf** statements, the object type of the base object is returned. For typeless objects such as scope names, type value “**Uninitialized**” is returned.

Table 15-14 Values Returned By the ObjectType Operator

Value	Meaning
0	Uninitialized
1	Integer
2	String
3	Buffer
4	Package
5	Field Unit
6	Device
7	Event
8	Method
9	Mutex
10	Operation Region
11	Power Resource
12	Processor
13	Thermal Zone
14	Buffer Field
15	DDB Handle
16	Debug Object
>16	Reserved

15.2.3.4.2.29 Or - Bit-wise Or

```
OrTerm              := Or(  
                        Source1,                               //TermArg=>Integer  
                        Source2                               //TermArg=>Integer  
                        Result                                //Nothing | SuperName  
                    ) => Integer
```

Source1 and *Source2* are evaluated as integer data types, a bit-wide **OR** is performed, and the result is optionally stored in *Result*.

15.2.3.4.2.30 RefOf - Reference Of

```
RefOfTerm           := RefOf(  
                        Object                               //SuperName  
                    ) => ObjectReference
```

Returns an object reference to *Object*. *Object* can be any object type (for example, a package, a device object, and so on).

If the *Object* does not exist, the result of a **RefOf** operation is fatal. Use the **CondRefOf** term in cases where the *Object* might not exist.

The primary purpose of RefOf() is to allow an object to be passed to a method as an argument to a method without the object being evaluated at the time of the method was loaded.

15.2.3.4.2.31 ShiftLeft - Shift Left

```
ShiftLeftTerm      := ShiftLeft(  
    Source,          //TermArg=>Integer  
    ShiftCount       //TermArg=>Integer  
    Result           //Nothing | SuperName  
) => Integer
```

Source and *ShiftCount* are evaluated as integer data types. *Source* is shifted left with the least significant bit zeroed *ShiftCount* times. The result is optionally stored into *Result*.

15.2.3.4.2.32 ShiftRight - Shift Right

```
ShiftRightTerm     := ShiftRight(  
    Source,          //TermArg=>Integer  
    ShiftCount       //TermArg=>Integer  
    Result           //Nothing | SuperName  
) => Integer
```

Source and *ShiftCount* are evaluated as integer data types. *Source* is shifted right with the most significant bit zeroed *ShiftCount* times. The result is optionally stored into *Result*.

15.2.3.4.2.33 SizeOf - SizeOf Data Object

```
SizeOfTerm         := SizeOf(  
    DataObject       //SuperName=>DataObject  
) => Integer
```

Returns the size of a buffer, string, or package data object. For a buffer it returns the size in bytes of the data. For a string, it returns the size in bytes of the string NOT counting the trailing NULL. For a package, it returns the number of elements.

15.2.3.4.2.34 Store - Store

```
StoreTerm          := Store(  
    Source,          //TermArg=>DataObject  
    Destination      //SuperName  
) => DataObject
```

This operation evaluates *Source* converts to the data type of *Destination* and writes the results into *Destination*. If the *Destination* is of the type Uninitialized, then the *Destination* object is initialized as shown in the following table.

Table 15-15 Store Operator Initialization Data Types for Uninitialized Destinations

Data Type	Description
Integer	Destination initialized as integer.
Buffer	Destination initialized as buffer.
String	Destination initialized as string.

The Buffer data type is a fixed length data type. If the source argument has a greater length than the destination size, extra data are truncated. If the source argument has a smaller length than the destination size, the rest of the destination data are zeroed. Stores to Operational Region Field data types may relinquish the processor depending on the region type.

All stores (of any type) to the constant zero, constant one, or constant ones object are not allowed. Stores to read-only objects are fatal. The execution result of the operation is the same as the data written to *Destination*.

The following example creates the name CNT that references an integer data object with the value 5 and then stores CNT to **Local0**. After the Store operation, **Local0** is an integer object with the value 5.

```
Name(CNT, 5)  
Store(CNT, Local0)
```

15.2.3.4.2.35 Subtract - Subtract

```
SubtractTerm      := Subtract(
                    Addend1,           //TermArg=>Integer
                    Addend2,           //TermArg=>Integer
                    Result              //Nothing | SuperName
                    ) => Integer
```

Addend1 and *Addend2* are evaluated as integer data types. *Addend2* is subtracted from *Addend1*, and the result is optionally stored into *Result*. Underflow conditions are ignored.

15.2.3.4.2.36 ToBCD - Convert to BCD

```
ToBCDTerm         := ToBCD(
                    Value,              //TermArg=>Integer
                    Result              //Nothing | SuperName
                    ) => Integer
```

The **ToBCD** operation is used to convert *Value* from a numeric format to a BCD format and optionally store the numeric value in *Result*.

15.2.3.4.2.37 Wait - Wait for a Synchronization Event

```
WaitTerm          := Wait(
                    SyncObject,         //SuperName
                    TimeoutValue        //TermArg=>Integer
                    ) => Boolean
```

SyncObject must be an event synchronization object. The calling method blocks waiting for the event to be signaled.

The pending signal count is decremented. If there is no pending signal count, the processor is relinquished until a signal count is posted to the Event or until at least *TimeoutValue* milliseconds have elapsed.

This operation returns a non-zero value if a timeout occurred and a signal was not acquired. A *TimeoutValue* of 0xFFFF indicates that there is no time out and the operation will wait indefinitely.

15.2.3.4.2.38 XOr - Bitwise XOR

```
XOrTerm           := XOr(
                    Source1,           //TermArg=>Integer
                    Source2,           //TermArg=>Integer
                    Result              //Nothing | SuperName
                    ) => Integer
```

Source1 and *Source2* are evaluated as integer data types, a bit-wise **XOR** is performed, and the result is optionally stored in *Result*.

15.2.3.5 User Terms

```
UserTerm          := NameString(
                    ArgList
                    ) => DataObject
```

NameString must be referring to an existing method object in the Name Space. It can either be an absolute Name Space path or else it must be accessible at the current scope of invocation. The number of arguments in *ArgList* must match the number of arguments declared in the method object.

15.2.3.6 Data Objects

There are four different types of data objects:

- Buffer terms
- Package terms
- Literal data terms
- Data macros

15.2.3.6.1 Buffer – Declare Buffer Object

```
BufferTerm      := Buffer(  
                    BuffSize                                //Nothing |  
                                                            //TermArg=>Integer  
                    ) {String | ByteList}
```

Declares a Buffer, of size *BuffSize* and initial value of *Initializer* (ByteList).

The optional *BuffSize* parameter specifies the size of the buffer and the initial value is specified in *Initializer* ByteList. If *BuffSize* is not specified, it defaults to the size of initializer. If the count is too small to hold the value specified by initializer, initializer size is used. For example, all four of the following examples generate the same datum in name space, although they have different ASL encodings:

```
Buffer(10) {"P00.00A"}  
Buffer(Arg0) {0x50 0x30 0x30 0x2e 0x30 0x30 0x41}  
Buffer(10) {0x50 0x30 0x30 0x2e 0x30 0x30 0x41 0x00 0x00 0x00}  
Buffer() {0x50 0x30 0x30 0x2e 0x30 0x30 0x41 0x00 0x00 0x00}
```

15.2.3.6.2 Package – Declare Package Object

```
PackageTerm     := Package(  
                    NumElements                            //Nothing |  
                                                            //ByteConst  
                    ) {PackageList}
```

Declares an unnamed aggregation of data items, constants, and/or references to control methods. The size of the package is *NumElements*. PackageList contains the list data items, constants, and/or control method references used to initialize the package. If *NumElements* is absent, it is set to match the number of elements in the PackageList. If *NumElements* is present and greater than the number of elements in the PackageList, the default entry Undefined is used to initialize the package elements beyond those initialized from the PackageList. Evaluating an undefined element will yield an error, but they can be assigned values to make them defined. It is an error for *NumElements* to be less than the number of elements in the PackageList

There are two types of package elements in the PackageList: data objects and references to control methods.

Note: If non method code package objects are implemented in an ASL compiler, evaluations of these objects are performed within the scope of the invoking method, and are performed when the containing definition block is loaded. This means that the targets of all stores, loads, and references to the locals, arguments, or constant terms are in the same name scope as the invoking method.

Example 1:

```
Package () {  
    3,  
    9,  
    "ACPI 1.0 COMPLIANT",  
    Package () {  
        "Checksum=>",  
        Package () {  
            7,  
            9  
        }  
    },  
    0  
}
```

Example 2: This example defines and initializes a two-dimensional array.

```
Package () {  
    Package () {11, 12, 13},  
    Package () {21, 22, 23}  
}
```

Example 3: This example is a legal encoding, but of no apparent use.

```
Package () {}
```

Example 4: This encoding allocates space for ten things to be defined later (see the **Name** and **Index** term definitions).

```
Package (10) {}
```

15.2.3.6.3 Literal Data Terms

Literal Data terms include:

- Integers
- Strings
- Constant data terms

15.2.3.6.3.1 Integers

```
Integer          := ByteConst | WordConst | DWordConst
ByteConst        := 0x00-0xff
WordConst        := 0x0000-0xffff
DWordConst       := 0x00000000-0xffffffff
```

Using the above grammar to define an object containing the value of integer causes the ASL compiler to automatically generate the proper width of the defined integer (Byte, Word, or DWord).

15.2.3.6.3.2 Strings

```
String           := ''' AsciiCharList '''
AsciiCharList    := Nothing | <AsciiChar AsciiCharList>
AsciiChar        := 0x01-0x7f
NullChar         := 0x00
```

The above grammar can be used to define an object containing a read-only string value. The default string value is the null string, which has 0 bytes available for storage of other values.

Since literal strings are read-only constants, the following ASL statement (for example) is not supported:

```
Store("ABC", "DEF")
```

However, the following sequence of statements is supported:

```
Name(STR, "DEF")
...

Store("ABC", STR)
```

15.2.3.6.3.3 Constant Data Terms

```
ConstTerm        := Zero | One | Ones | Revision
```

The constant declaration terms are **Zero**, **One**, **Ones**, and **Revision**.

15.2.3.6.3.3.1 Zero - Constant Zero Object

The constant **Zero** object is an object of type Integer that will always read as all bits clear. Writes to this object are not allowed.

15.2.3.6.3.3.2 One - Constant One Object

The constant **One** object is an object of type Integer that will always read the LSb as set and all other bits as clear (that is, the value of 1). Writes to this object are not allowed.

15.2.3.6.3.3.3 Ones - Constant Ones Object

The constant **Ones** object is an object of type Integer that will always read as all bits set. Writes to this object are not allowed.

15.2.3.6.3.3.4 Revision – Constant Revision Object

The constant **Revision** object is an object of type Integer that will always read as the revision of the AML interpreter.

15.2.3.6.4 Data Macors

The data macros are:

- EISAID terms.
- ResourceTemplate terms.

15.2.3.6.4.1 EISAID Macro - Convert EISA ID String To Integer

```
EISAIDTerm          := EISAID(  
                        EISAIDString          //String  
                        ) => DWordConst
```

Converts *EISAIDString*, a 7-character text string argument, into its corresponding 4-byte numeric EISA ID encoding. The can be used when declaring IDs for devices that have EISA IDs.

15.2.3.6.4.2 ResourceTemplate Macro – Convert Resource To Buffer Format

```
ResourceTemplateTerm := ResourceTemplate() {ResourceMacroList} => BufferTerm
```

For a full definition of the ResourceTemplateTerm macro, see section 6.4.1.

15.2.3.7 Miscellaneous Objects

Miscellaneous objects include:

- Debug objects
- ArgX objects
- LocalX objects

15.2.3.7.1 Debug Data Object

```
DebugTerm          := Debug
```

The debug data object is a virtual data object. Writes to this object provide debugging information. On at least debug versions of the interpreter any writes into this object are appropriately displayed on the system's native kernel debugger. All writes to the debug object are otherwise benign. If the system is in use without a kernel debugger, then writes to the debug object are ignored. The following table relates the ASL term types that can be written to the Debug object to the format of the information on the kernel debugger display.

Table 15-16 Debug Object Display Formats

ASL Term Type	Display Format
Numeric data object	All digits displayed in hexadecimal format.
String data object	String is displayed
Object reference	Information about the object is displayed (for example, object type and object name), but the object is not evaluated.

The Debug object is a write-only object; attempting to read from the debug object is not supported.

15.2.3.7.2 ArgX – Argument Data Objects

```
ArgTerm          := Arg0 | Arg1 | Arg2 | Arg3 | Arg4 | Arg5 | Arg6
```

Up to 7 argument object references can be passed to a control method. On entry to a control method, only the argument objects that are passed are usable.

15.2.3.7.3 LocalX - Local Data Objects

```
LocalTerm          := Local0 | Local1 | Local2 | Local3 | Local4 | Local5 | Local6  
                    | Local7
```

Up to 8 local objects can be referenced in a control method. On entry to a control method these objects are uninitialized and cannot be used until some value or reference is stored into the object. Once initialized, these objects are preserved in the scope of execution for that control method.

16. ACPI Machine Language (AML) Specification

This section formally defines the ACPI Control Method Machine Language (AML) language. AML is the ACPI Control Method virtual machine language, machine code for a virtual machine which is supported by an ACPI-compatible OS. ACPI control methods can be written in AML, but humans ordinarily write control methods in ASL.

AML is the language processed by the ACPI method interpreter. It is primarily a declarative language. It's best not to think of it as a stream of code, but rather as a set of declarations that the ACPI interpreter will compile into the ACPI name space at definition block load time. For example, notice that DefByte allocates an anonymous integer variable with a byte size initial value in ACPI space, and passes in an initial value. The byte in the AML stream that defines the initial value is *not* the address of the variable's storage location.

An OEM or BIOS vendor needs to write ASL and be able to single step AML for debugging. (Debuggers and other ACPI control method language tools are expected to be AML level tools, not source level tools.) An ASL translator implementer must understand how to read ASL and generate AML. An AML interpreter author must understand how to execute AML.

AML and ASL are *different languages* though they are closely related.

All ACPI-compatible OSes must support AML. A given user can define some arbitrary source language (to replace ASL) and write a tool to translate it to AML. However, the ACPI group will support a single translator for a single language, ASL.

16.1 Notation Conventions

The notation conventions in the table below help the reader to interpret the AML formal grammar.

Table 16-1 AML Grammar Notation Conventions

Notation Convention	Description	Example
0xdd	Refers to a byte value expressed as 2 hexadecimal digits.	0x21
Number in bold.	Denotes the encoding of the AML term.	
Term => Evaluated Type	Shows the resulting type of the evaluation of Term.	
Single quotes (' ')	Indicate constant characters.	'A' => 0x41
Term := Term Term ...	The term to the left of := can be expanded into the sequence of terms on the right.	aterm := bterm cterm means that aterm can be expanded into the two-term sequence of bterm followed by cterm.
Term Term Term ...	Terms separated from each other by spaces form an ordered list.	
Angle brackets (< >)	Used to group items.	<a b> <c d> means either a b or c d.
Bar symbol ()	Separates alternatives.	aterm := bterm [cterm dterm] means the following constructs are possible: bterm cterm dterm aterm := [bterm cterm] dterm means the following constructs are possible: bterm dterm

Notation Convention	Description	Example
		cterm dterm
Dash character (-)	Indicates a range.	1-9 means a single digit in the range 1 to 9 inclusive.
Parenthesized term following another term.	The parenthesized term is the repeat count of the previous term.	aterm(3) means aterm aterm aterm. bterm(N) means N number of bterms.

16.2 AML Grammar Definition

This section defines the byte values that make up an AML byte stream.

```

AMLCode                := DefBlockHdr TermList
DefBlockHdr            := TableSig TableLen SpecCompliance CheckSum OemID
                        OemTableID OemRev CreatorID CreatorRev
TableSig               := DWordConst
                        //As defined in section 5.2.3.
TableLen               := DWordConst
                        //Length of the table in bytes including the block
                        //header.
SpecCompliance         := ByteConst
                        //The revision of the structure.
CheckSum               := ByteConst
                        //Byte checksum of the entire table.
OemID                  := ByteConst(6)
                        //OEM ID of up to 6 characters. If the OEM ID is
                        //shorter than 6 characters, it can be terminated
                        //with a NULL character.
OemTableID             := ByteConst(8)
                        //OEM Table ID of up to 8 characters. If the OEM
                        //Table ID is shorter than 8 characters, it can be
                        //terminated with a NULL character.
OemRev                 := DWordConst
                        //OEM Table Revision.
CreatorID              := DWordConst
                        //Vendor ID of the ASL assembler. For example,
                        //the Microsoft provided assembler has a vendor
                        //ID "MSFT".
CreatorRev             := DWordConst
                        //Revision of the ASL assembler.

```

The AML encoding can be categorized in the following groups:

- Name objects encoding
- Data objects encoding
- Package length encoding
- Term objects encoding
- Miscellaneous objects encoding

16.2.1 Name Objects Encoding

```

LeadNameChar          := 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G' | 'H' | 'I' | 'J'
                        | 'K' | 'L' | 'M' | 'N' | 'O' | 'P' | 'Q' | 'R' | 'S' |
                        'T' | 'U' | 'V' | 'W' | 'X' | 'Y' | 'Z' | '_'
NameChar               := '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
                        | LeadNameChar
RootChar               := '\\'
ParentPrefixChar       := '^'

'A'-'Z'               := 0x41-0x5a
'_'                   := 0x5f
'0'-'9'               := 0x30-0x39
'\\'                  := 0x5c
'^'                   := 0x5e

NameSeg                := <LeadNameChar NameChar NameChar NameChar>
                        // Note that NameSegs shorter than 4 characters are
                        // filled with trailing '_'s.
NameString             := <RootChar NamePath> | <PrefixPath NamePath>
PrefixPath             := Nothing | <'^' PrefixPath>
NamePath               := NullName | NameSeg | DualNamePath | MultiNamePath

```

```

NullName           := 0x00
DualNamePath       := DualNamePrefix NameSeg NameSeg
DualNamePrefix     := 0x2e
MultiNamePath      := MultiNamePrefix SegCount NameSeg (SegCount)
MultiNamePrefix    := 0x2f
SegCount           := ByteData
                  // SegCount can be from 1 to 255.
                  // MultiNamePrefix(35) => 0x2f 0x23
                  // and following by 35 NameSegs.
                  // So, the total encoding length
                  // will be 1 + 1 + 35*4 = 142.
                  // Note that:
                  //   DualNamePrefix NameSeg NameSeg
                  // has a smaller encoding than the
                  // equivalent encoding of:
                  //   MultiNamePrefix(2) NameSeg NameSeg
SuperName          := NameString | ArgObj | LocalObj | DebugObj | DefIndex

```

16.2.2 Data Objects Encoding

```

DataObject         := LiteralData | DefBuffer | DefPackage
DataObjectList     := Nothing | <DataObject DataObjectList>
LiteralData        := ByteConst | WordConst | DWordConst | String | ConstObj |
                  RevisionOp
ComputationalData  := LiteralData | DefBuffer
ByteConst          := BytePrefix ByteData
BytePrefix         := 0x0a
WordConst          := WordPrefix WordData
WordPrefix         := 0x0b
DWordConst         := DWordPrefix DWordData
DWordPrefix        := 0x0c
String             := StringPrefix AsciiCharList NullChar
StringPrefix       := 0x0d
ConstObj           := ZeroOp | OneOp | OnesOp
ByteList           := Nothing | <ByteData ByteList>
ByteData           := 0x00-0xff
WordData           := ByteData ByteData
                  // 0x0000-0xffff
DWordData          := ByteData ByteData ByteData ByteData
                  // 0x00000000-0xffffffff
AsciiCharList      := Nothing | <AsciiChar AsciiCharList>
AsciiChar          := 0x01-0x7f
NullChar           := 0x00
ZeroOp             := 0x00
OneOp              := 0x01
OnesOp             := 0xff
RevisionOp         := ExtOpPrefix 0x30
ExtOpPrefix        := 0x5b

```

16.2.3 Package Length Encoding

```

PkgLength          := PkgLeadByte |
                  <PkgLeadByte ByteData> |
                  <PkgLeadByte ByteData ByteData> |
                  <PkgLeadByte ByteData ByteData ByteData>
PkgLeadByte        := <bit 7-6: follow ByteData count>
                  <bit 5-4: reserved>
                  <bit 3-0: least significant package length byte>
                  // Note: The high 2 bits of the first byte reveal how
                  // many follow bytes are in the PkgLength. If the
                  // PkgLength has only one byte, bit 0 through 5 are
                  // used to encode the package length (i.e. values
                  // 0-63). If the package length value is more than
                  // 63, more than one byte must be used for the
                  // encoding in which case bit 5 and 4 of the
                  // PkgLeadByte are reserved and must be zero. If
                  // multiple bytes encoding is used, bits 3-0 of the
                  // PkgLeadByte become the least significant 4 bits
                  // of the resulting package length value. The next
                  // ByteData will become the next least significant
                  // 8 bits of the resulting value and so on.

```

16.2.4 Term Objects Encoding

```
TermObj          := NameSpaceModifierObj | NamedObj | Type1Opcode |  
                  Type2Opcode | UserTermObj  
TermList         := Nothing | <TermObj TermList>  
  
TermArg          := Type2Opcode | DataObject | UserTermObj | ArgObj |  
                  LocalObj  
UserTermObj      := NameString TermArgList  
TermArgList      := Nothing | <TermArg TermArgList>  
  
ObjectList       := Nothing | <Object ObjectList>  
Object           := NameSpaceModifierObj | NamedObj
```

16.2.4.1 Name Space Modifier Objects Encoding

```
NameSpaceModifierObj := DefAlias | DefName | DefScope  
  
DefAlias             := AliasOp NameString NameString  
AliasOp              := 0x06  
  
DefName              := NameOp NameString DataObject  
NameOp               := 0x08  
  
DefScope             := ScopeOp PkgLength NameString TermList  
ScopeOp              := 0x10
```

16.2.4.2 Named Objects Encoding

```
NamedObj           := DefBankField | DefCreateBitField | DefCreateByteField |  
                    DefCreateDWordField | DefCreateField |  
                    DefCreateWordField | DefDevice | DefEvent | DefField |  
                    DefIndexField | DefMethod | DefMutex | DefOpRegion |  
                    DefPowerRes | DefProcessor | DefThermalZone  
  
DefBankField       := BankFieldOp PkgLength NameString NameString BankValue  
                    FieldFlags FieldList  
BankFieldOp        := ExtOpPrefix 0x87  
BankValue          := TermArg=>Integer  
FieldFlags         := ByteData  
                    // bit 0-3: AccessType  
                    // 0: AnyAcc  
                    // 1: ByteAcc  
                    // 2: WordAcc  
                    // 3: DWordAcc  
                    // 4: BlockAcc  
                    // 5: SMBSendRecvAcc  
                    // 6: SMBQuickAcc  
                    // bit 4: LockRule  
                    // 0: NoLock  
                    // 1: Lock  
                    // bit 5-6: UpdateRule  
                    // 0: Preserve  
                    // 1: WriteAsOnes  
                    // 2: WriteAsZeros  
                    // bit 7: reserved (must be 0)  
FieldList          := Nothing | <FieldElement FieldList>  
FieldElement       := NamedField | ReservedField | AccessField  
NamedField         := NameSeg PkgLength  
ReservedField      := 0x00 PkgLength  
AccessField        := 0x01 AccessType AccessAttrib  
AccessType         := ByteData  
                    // Same as AccessType bits of FieldFlags  
AccessAttrib       := ByteData  
  
DefCreateBitField   := CreateBitFieldOp SourceBuff BitIndex NameString  
CreateBitFieldOp    := 0x8d  
SourceBuff         := TermArg=>BufferObj  
BitIndex           := TermArg=>Integer  
  
DefCreateByteField   := CreateByteFieldOp SourceBuff ByteIndex NameString  
CreateByteFieldOp    := 0x8c  
ByteIndex          := TermArg=>Integer
```

DefCreateDWordField	:= CreateDWordFieldOp SourceBuff ByteIndex NameString
CreateDWordFieldOp	:= 0x8a
DefCreateField	:= CreateFieldOp SourceBuff BitIndex NumBits NameString
CreateFieldOp	:= ExtOpPrefix 0x13
NumBits	:= TermArg=>Integer
DefCreateWordField	:= CreateWordFieldOp SourceBuff ByteIndex NameString
CreateWordFieldOp	:= 0x8b
DefDevice	:= DeviceOp PkgLength NameString ObjectList
DeviceOp	:= ExtOpPrefix 0x82
DefEvent	:= EventOp NameString
EventOp	:= ExtOpPrefix 0x02
DefField	:= FieldOp PkgLength NameString FieldFlags FieldList
FieldOp	:= ExtOpPrefix 0x81
DefIndexField	:= IndexFieldOp PkgLength NameString NameString FieldFlags
IndexFieldOp	:= ExtOpPrefix 0x86
DefMethod	:= MethodOp PkgLength NameString MethodFlags TermList
MethodOp	:= 0x14
MethodFlags	:= ByteData // bit 0-2: ArgCount (0-7) // bit 3: SerializeFlag // 0: NotSerialized // 1: Serialized // bit 4-7: reserved (must be 0)
DefMutex	:= MutexOp NameString SyncFlags
MutexOp	:= ExtOpPrefix 0x01
SyncFlags	:= ByteData // bit 0-3: SyncLevel (0x00-0x0f) // bit 4-7: reserved (must be 0)
DefOpRegion	:= OpRegionOp NameString RegionSpace RegionOffset RegionLen
OpRegionOp	:= ExtOpPrefix 0x80
RegionSpace	:= ByteData // 0x00: SystemMemory // 0x01: SystemIO // 0x02: PCI_Config // 0x03: EmbeddedControl // 0x04: SMBus // 0x80-0xff: user defined
RegionOffset	:= TermArg=>DWordData
RegionLen	:= TermArg=>DWordData
DefPowerRes	:= PowerResOp PkgLength NameString SystemLevel ResourceOrder
PowerResOp	:= ExtOpPrefix 0x84
SystemLevel	:= ByteData
ResourceOrder	:= WordData
DefProcessor	:= ProcessorOp PkgLength NameString ProcID PBlkAddr PBlkLen
ProcessorOp	:= ExtOpPrefix 0x83
ProcID	:= ByteData
PBlkAddr	:= DWordData
PBlkLen	:= ByteData
DefThermalZone	:= ThermalZoneOp PkgLength NameString ObjectList
ThermalZoneOp	:= ExtOpPrefix 0x85

16.2.4.3 Type 1 Opcodes Encoding

TypelOpcode	:= DefBreak DefBreakPoint DefFatal DefIfElse DefLoad DefNoop DefNotify DefRelease DefReset DefReturn DefSignal DefSleep DefStall DefUnload DefWhile
DefBreak	:= BreakOp

BreakOp	:= 0xa5
DefBreakPoint	:= BreakPointOp
BreakPointOp	:= 0xcc
DefElse	:= Nothing <ElseOp PkgLength TermList>
ElseOp	:= 0xa1
DefFatal	:= FatalOp FatalType FatalCode FatalArg
FatalOp	:= ExtOpPrefix 0x32
FatalType	:= ByteData
FatalCode	:= DWordData
FatalArg	:= TermArg=>Integer
DefIfElse	:= IfOp PkgLength Predicate TermList DefElse
IfOp	:= 0xa0
Predicate	:= TermArg=>Integer
DefLoad	:= LoadOp NameString DDBHandleObject
LoadOp	:= ExtOpPrefix 0x20
DDBHandleObject	:= SuperName
DefNoop	:= NoopOp
NoopOp	:= 0xa3
DefNotify	:= NotifyOp NotifyObject NotifyValue
NotifyOp	:= 0x86
NotifyObject	:= SuperName
NotifyValue	:= TermArg=>Integer
DefRelease	:= ReleaseOp MutexObject
ReleaseOp	:= ExtOpPrefix 0x27
MutexObject	:= SuperName
DefReset	:= ResetOp EventObject
ResetOp	:= ExtOpPrefix 0x26
EventObject	:= SuperName
DefReturn	:= ReturnOp ArgObject
ReturnOp	:= 0xa4
ArgObject	:= TermArg=>DataObject
DefSignal	:= SignalOp EventObject
SignalOp	:= ExtOpPrefix 0x24
DefSleep	:= SleepOp MSecTime
SleepOp	:= ExtOpPrefix 0x22
MSecTime	:= TermArg=>Integer
DefStall	:= StallOp USecTime
StallOp	:= ExtOpPrefix 0x21
USecTime	:= TermArg=>ByteData
DefUnload	:= UnloadOp DDBHandleObject
UnloadOp	:= ExtOpPrefix 0x2a
DefWhile	:= WhileOp PkgLength Predicate TermList
WhileOp	:= 0xa2

16.2.4.4 Type 2 Opcodes Encoding

Type2Opcode	:= DefAcquire DefAdd DefAnd DefBuffer DefConcat DefCondRefOf DefDecrement DefDerefOf DefDivide DefFindSetLeftBit DefFindSetRightBit DefFromBCD DefIncrement DefIndex DefLAnd DefLEqual DefLGreater DefLGreaterEqual DefLLess DefLLessEqual DefLNot DefLNotEqual DefLor DefMatch DefMultiply DefNAnd DefNOr DefNot DefObjectType DefOr DefPackage DefRefOf DefShiftLeft DefShiftRight DefSizeOf DefStore DefSubtract DefToBCD DefWait DefXOr
DefAcquire	:= AcquireOp MutexObject Timeout
AcquireOp	:= ExtOpPrefix 0x23

Timeout	:= WordData
DefAdd	:= AddOp Operand1 Operand2 Target
AddOp	:= 0x72
Operand1	:= TermArg=>Integer
Operand2	:= TermArg=>Integer
Target	:= SuperName NullName
DefAnd	:= AndOp Operand1 Operand2 Target
AndOp	:= 0x7b
DefBuffer	:= BufferOp PkgLength BufferSize ByteList
BufferOp	:= 0x11
BufferSize	:= TermArg=>Integer
DefConcat	:= ConcatOp Data1 Data2 Target
ConcatOp	:= 0x73
Data1	:= TermArg=>ComputationalData
Data2	:= TermArg=>ComputationalData
DefCondRefOf	:= CondRefOfOp SuperName SuperName
CondRefOfOp	:= ExtOpPrefix 0x12
DefDecrement	:= DecrementOp SuperName
DecrementOp	:= 0x76
DefDerefOf	:= DerefOfOp ObjReference
DerefOfOp	:= 0x83
ObjReference	:= TermArg=>ObjectReference //ObjectReference is an object produced by terms //such as Index, RefOf or CondRefOf.
DefDivide	:= DivideOp Dividend Divisor Remainder Quotient
DivideOp	:= 0x78
Dividend	:= TermArg=>Integer
Divisor	:= TermArg=>Integer
Remainder	:= Target
Quotient	:= Target
DefFindSetLeftBit	:= FindSetLeftBitOp Operand Target
FindSetLeftBitOp	:= 0x81
Operand	:= TermArg=>Integer
DefFindSetRightBit	:= FindSetRightBitOp Operand Target
FindSetRightBitOp	:= 0x82
DefFromBCD	:= FromBCDOp BCDValue Target
FromBCDOp	:= ExtOpPrefix 0x28
BCDValue	:= TermArg=>Integer
DefIncrement	:= IncrementOp SuperName
IncrementOp	:= 0x75
DefIndex	:= IndexOp BuffPkgObj IndexValue Target
IndexOp	:= 0x88
BuffPkgObj	:= TermArg=>Buffer or Package object
IndexValue	:= TermArg=>Integer
DefLAnd	:= LAndOp Operand1 Operand2
LAndOp	:= 0x90
DefLEqual	:= LEqualOp Operand1 Operand2
LEqualOp	:= 0x93
DefLGreater	:= LGreaterOp Operand1 Operand2
LGreaterOp	:= 0x94
DefLGreaterEqual	:= LGreaterEqualOp Operand1 Operand2
LGreaterEqualOp	:= LNotOp LLessOp
DefLLess	:= LLessOp Operand1 Operand2
LLessOp	:= 0x95
DefLLessEqual	:= LLessEqualOp Operand1 Operand2

LLessEqualOp	:= LNotOp LGreaterOp
DefLNot	:= LNotOp Operand
LNotOp	:= 0x92
DefLNotEqual	:= LNotEqualOp Operand1 Operand2
LNotEqualOp	:= LNotOp LEqualOp
DefLOr	:= LOrOp Operand1 Operand2
LOrOp	:= 0x91
DefMatch	:= MatchOp SearchPkg Opcode1 Operand1 Opcode2 Operand2 StartIndex
MatchOp	:= 0x89
SearchPkg	:= TermArg=>PackageObject
Opcode1	:= ByteData // 0: MTR // 1: MEQ // 2: MLE // 3: MLT // 4: MGE // 5: MGT
Opcode2	:= ByteData (same as Opcode1)
StartIndex	:= TermArg=>Integer
DefMultiply	:= MultiplyOp Operand1 Operand2 Target
MultiplyOp	:= 0x77
DefNAnd	:= NAndOp Operand1 Operand2 Target
NAndOp	:= 0x7c
DefNOr	:= NOrOp Operand1 Operand2 Target
NOrOp	:= 0x7e
DefNot	:= NotOp Operand Target
NotOp	:= 0x80
DefObjectType	:= ObjectTypeOp SuperName
ObjectTypeOp	:= 0x8e
DefOr	:= OrOp Operand1 Operand2 Target
OrOp	:= 0x7d
DefPackage	:= PackageOp PkgLength NumElements PackageElementList
PackageOp	:= 0x12
NumElements	:= ByteData
PackageElementList	:= Nothing <PackageElement PackageElementList>
PackageElement	:= DataObject NameString
DefRefOf	:= RefOfOp SuperName
RefOfOp	:= 0x71
DefShiftLeft	:= ShiftLeftOp Operand ShiftCount Target
ShiftLeftOp	:= 0x79
ShiftCount	:= TermArg=>Integer
DefShiftRight	:= ShiftRightOp Operand ShiftCount Target
ShiftRightOp	:= 0x7a
DefSizeOf	:= SizeOfOp SuperName
SizeOfOp	:= 0x87
DefStore	:= StoreOp Operand SuperName
StoreOp	:= 0x70
DefSubtract	:= SubtractOp Operand1 Operand2 Target
SubtractOp	:= 0x74
DefToBCD	:= ToBCDOp Operand Target
ToBCDOp	:= ExtOpPrefix 0x29
DefWait	:= WaitOp EventObject Timeout
WaitOp	:= ExtOpPrefix 0x25

```

DefXOr      := XOrOp Operand1 Operand2 Target
XOrOp      := 0x7f

```

16.2.5 Miscellaneous Objects Encoding

Miscellaneous objects include:

- Arg objects
- Local objects
- Debug objects

16.2.5.1 Arg Objects Encoding

```

ArgObj      := Arg0Op | Arg1Op | Arg2Op | Arg3Op | Arg4Op | Arg5Op |
              Arg6Op
Arg0Op      := 0x68
Arg1Op      := 0x69
Arg2Op      := 0x6a
Arg3Op      := 0x6b
Arg4Op      := 0x6c
Arg5Op      := 0x6d
Arg6Op      := 0x6e

```

16.2.5.2 Local Objects Encoding

```

LocalObj    := Local0Op | Local1Op | Local2Op | Local3Op | Local4Op |
              Local5Op | Local6Op | Local7Op
Local0Op    := 0x60
Local1Op    := 0x61
Local2Op    := 0x62
Local3Op    := 0x63
Local4Op    := 0x64
Local5Op    := 0x65
Local6Op    := 0x66
Local7Op    := 0x67

```

16.2.5.3 Debug Objects Encoding

```

DebugObj    := DebugOp
DebugOp     := ExtOpPrefix 0x31

```

16.3 AML Byte Stream Byte Values

The following table lists all the byte values that can be found in an AML byte stream and the meaning of each byte value. This table is useful for debugging AML code.

Table 16-2 AML Byte Stream Byte Values

Encoding Value	Encoding Name	Encoding Group	Fixed List Arguments	Variable List Arguments
0x00	ZeroOp	Data Object	--	--
0x01	OneOp	Data Object	--	--
0x02-0x05	--	--	--	--
0x06	AliasOp	Term Object	NameString NameString	--
0x07	--	--	--	--
0x08	NameOp	Term Object	NameString DataObject	--
0x09	--	--	--	--
0x0A	BytePrefix	Data Object	ByteData	--
0x0B	WordPrefix	Data Object	WordData	--
0x0C	DwordPrefix	Data Object	DWordData	--
0x0D	StringPrefix	Data Object	AsciiCharList NullChar	--
0x0E-0x0F	--	--	--	--

Encoding Value	Encoding Name	Encoding Group	Fixed List Arguments	Variable List Arguments
0x10	ScopeOp	Term Object	NameString	TermList
0x11	BufferOp	Term Object	TermArg	ByteList
0x12	PackageOp	Term Object	ByteData	DataObjectList
0x13	--	--	--	--
0x14	MethodOp	Term Object	NameString ByteData	TermList
0x15-0x2D	--	--	--	--
0x2E (`.')`)	DualNamePrefix	Name Object	NameSeg NameSeg	--
0x2F (`/'`)	MultiNamePrefix	Name Object	ByteData NameSeg (N)	--
0x30-0x40	--	--	--	--
0x41-0x5A (`A' - `Z'`)	NameChar	Name Object	--	--
0x5B (`[` `)`	ExtOpPrefix	--	ByteData	--
0x5B 0x01	MutexOp	Term Object	NameString ByteData	--
0x5B 0x02	EventOp	Term Object	NameString	--
0x5B 0x12	CondRefOfOp	Term Object	SuperName SuperName	--
0x5B 0x13	CreateFieldOp	Term Object	TermArg TermArg TermArg NameString	--
0x5B 0x20	LoadOp	Term Object	NameString SuperName	--
0x5B 0x21	StallOp	Term Object	TermArg	--
0x5B 0x22	SleepOp	Term Object	TermArg	--
0x5B 0x23	AcquireOp	Term Object	SuperName WordData	--
0x5B 0x24	SignalOp	Term Object	SuperName	--
0x5B 0x25	WaitOp	Term Object	SuperName TermArg	--
0x5B 0x26	ResetOp	Term Object	SuperName	--
0x5B 0x27	ReleaseOp	Term Object	SuperName	--
0x5B 0x28	FromBCDOp	Term Object	TermArg Target	--
0x5B 0x29	ToBCD	Term Object	TermArg Target	--
0x5B 0x2A	UnloadOp	Term Object	SuperName	--
0x5B 0x30	RevisionOp	Data Object	--	--
0x5B 0x31	DebugOp	Debug Object	--	--
0x5B 0x32	FatalOp	Term Object	ByteData DWordData TermArg	--
0x5B 0x80	OpRegionOp	Term Object	NameString ByteData TermArg TermArg	--
0x5B 0x81	FieldOp	Term Object	NameString ByteData	FieldList
0x5B 0x82	DeviceOp	Term Object	NameString	ObjectList
0x5B 0x83	ProcessorOp	Term Object	NameString ByteData DWordData ByteData	ObjectList
0x5B 0x84	PowerResOp	Term Object	NameString ByteData WordData	ObjectList
0x5B 0x85	ThermalZoneOp	Term Object	NameString	ObjectList
0x5B 0x86	IndexFieldOp	Term Object	NameString NameString ByteData	FieldList

Encoding Value	Encoding Name	Encoding Group	Fixed List Arguments	Variable List Arguments
0x5B 0x87	BankFieldOp	Term Object	NameString NameString TermArg ByteData	FieldList
0x5C ('\')	RootChar	Name Object	--	--
0x5D	--	--	--	--
0x5E (^)	ParentPrefixChar	Name Object	--	--
0x5F	--	--	--	--
0x60 (')	Local0Op	Local Object	--	--
0x61 ('a')	Local1Op	Local Object	--	--
0x62 ('b')	Local2Op	Local Object	--	--
0x63 ('c')	Local3Op	Local Object	--	--
0x64 ('d')	Local4Op	Local Object	--	--
0x65 ('e')	Local5Op	Local Object	--	--
0x66 ('f')	Local6Op	Local Object	--	--
0x67 ('g')	Local7Op	Local Object	--	--
0x68 ('h')	Arg0Op	Arg Object	--	--
0x69 ('i')	Arg1Op	Arg Object	--	--
0x6A ('j')	Arg2Op	Arg Object	--	--
0x6B ('k')	Arg3Op	Arg Object	--	--
0x6C ('l')	Arg4Op	Arg Object	--	--
0x6D ('m')	Arg5Op	Arg Object	--	--
0x6E ('n')	Arg6Op	Arg Object	--	--
0x6F	--	--	--	--
0x70	StoreOp	Term Object	TermArg SuperName	--
0x71	RefOfOp	Term Object	SuperName	--
0x72	AddOp	Term Object	TermArg TermArg Target	--
0x73	ConcatOp	Term Object	TermArg TermArg Target	--
0x74	SubtractOp	Term Object	TermArg TermArg Target	--
0x75	IncrementOp	Term Object	SuperName	--
0x76	DecrementOp	Term Object	SuperName	--
0x77	MultiplyOp	Term Object	TermArg TermArg Target	--
0x78	DivideOp	Term Object	TermArg TermArg Target Target	--
0x79	ShiftLeftOp	Term Object	TermArg TermArg Target	--
0x7A	ShiftRightOp	Term Object	TermArg TermArg Target	--
0x7B	AndOp	Term Object	TermArg TermArg Target	--
0x7C	NAndOp	Term Object	TermArg TermArg Target	--
0x7D	OrOp	Term Object	TermArg TermArg Target	--
0x7E	NOrOp	Term Object	TermArg TermArg Target	--
0x7F	XOrOp	Term Object	TermArg TermArg Target	--
0x80	NotOp	Term Object	TermArg Target	--

Encoding Value	Encoding Name	Encoding Group	Fixed List Arguments	Variable List Arguments
0x81	FindSetLeftBitOp	Term Object	TermArg Target	--
0x82	FindSetRightBitOp	Term Object	TermArg Target	--
0x83	DerefOfOp	Term Object	TermArg	--
0x84-0x85	--	--	--	--
0x86	NotifyOp	Term Object	SuperName TermArg	--
0x87	SizeOfOp	Term Object	SuperName	--
0x88	IndexOp	Term Object	TermArg TermArg Target	--
0x89	MatchOp	Term Object	TermArg ByteData TermArg ByteData TermArg TermArg	--
0x8A	CreateDWordFieldOp	Term Object	TermArg TermArg NameString	--
0x8B	CreateWordFieldOp	Term Object	TermArg TermArg NameString	--
0x8C	CreateByteFieldOp	Term Object	TermArg TermArg NameString	--
0x8D	CreateBitFieldOp	Term Object	TermArg TermArg NameString	--
0x8E	ObjectTypeOp	Term Object	SuperName	--
0x8F	--	--	--	--
0x90	LAndOp	Term Object	TermArg TermArg	--
0x91	LOrOp	Term Object	TermArg TermArg	--
0x92	LNotOp	Term Object	TermArg	--
0x92 0x93	LNotEqualOp	Term Object	TermArg TermArg	--
0x92 0x94	LLessEqualOp	Term Object	TermArg TermArg	--
0x95 0x92	LGreaterEqualOp	Term Object	TermArg TermArg	--
0x93	LEqualOp	Term Object	TermArg TermArg	--
0x94	LGreaterOp	Term Object	TermArg TermArg	--
0x95	LLessOp	Term Object	TermArg TermArg	--
0x96-0x9F	--	--	--	--
0xA0	IfOp	Term Object	TermArg	TermList
0xA1	ElseOp	Term Object	--	TermList
0xA2	WhileOp	Term Object	TermArg	TermList
0xA3	NoopOp	Term Object	--	--
0xA4	ReturnOp	Term Object	TermArg	--
0xA5	BreakOp	Term Object	--	--
0xA6-0xCB	--	--	--	--
0xCC	BreakPointOp	Term Object	--	--
0xCD-0xFE	--	--	--	--
0xFF	OnesOp	Data Object	--	--

16.4 AML Encoding of Names in the Name Space

Assume the following name space exists:

```
\
  S0
    MEM
      SET
      GET
  S1
    MEM
      SET
      GET
    CPU
      SET
      GET
```

Assume further that a definition block is loaded that creates a node \S0.CPU.SET, and loads a block using it as a root. Assume the loaded block contains the following names:

```
STP1
^GET
^^PCI0
^^PCI0.SBS
\S2
\S2.ISA.COM1
^^S3
^^S2.MEM
^^S2.MEM.SET
Scope(\S0.CPU.SET.STP1) {
  XYZ
  ^ABC
  ^ABC.DEF
}
```

This will be encoded in AML as:

```
'STP1'
ParentPrefixChar 'GET_'
ParentPrefixChar ParentPrefixChar 'PCI0'
ParentPrefixChar ParentPrefixChar DualNamePrefix 'PCI0' 'SBS_'
RootChar 'S2__'
RootChar MultiNamePrefix 3 'S2__' 'ISA_' 'COM1'
ParentPrefixChar ParentPrefixChar ParentPrefixChar 'S3__'
ParentPrefixChar ParentPrefixChar ParentPrefixChar DualNamePrefix 'S2__' 'MEM_'
ParentPrefixChar ParentPrefixChar ParentPrefixChar MultiNamePrefix 3 'S2__' 'MEM_'
'SET_'
```

After the block is loaded, the name space will look like this (names added to the name space by the loading operation are shown in *italics*).

```

\
S0
  MEM
    SET
    GET
  CPU
    SET
      STP1
        XYZ
      ABC
      DEF
    GET
  PCI0
  SBS

S1
  MEM
    SET
    GET
  CPU
    SET
    GET

S2
  ISA
  COM1
  MEM
  SET

S3

```